

《计算机组成原理》

（第三讲）

厦门大学信息学院软件工程系 张海英

2026年3月

目录

- 第1章 计算机系统概论
- 第2章 数据信息的表示
- 第3章 运算方法与运算器
- 第4章 存储系统
- 第5章 指令系统
- 第6章 中央处理器
- 第7章 指令流水线
- 第8章 总线系统
- 第9章 输入输出系统



第3章 运算方法与运算器

- 3.1 计算机中的运算
- 3.2 定点加减法运算
- 3.3 定点乘法运算
- 3.4 定点除法运算
- 3.5 浮点运算
- 3.6 运算器

3.1 计算机中的运算

- 3.1.1 C语言中的位运算

- 位运算属于数值运算。

- ① & 与

- ② | 或

- ③ ~ 非（取反）

- ④ ^ 异或

3.1.1 C语言中的位运算

– C语言中位运算和汇编代码的对应关系:

– AT&T汇编语言

l表示long 32bit
w表示字16bit
b表示字节8bit

第一操作数为源
第二为目的
立即数加\$,寄存器加%

```
int i=1,j=-1;
```

```
0x40134e movl $0x1, 0xc(%esp)
```

```
0x401356 movl $0xffffffff, 0x8(%esp)
```

```
i=i&j;
```

```
0x40135e mov 0x8(%esp), %eax
```

```
0x401362 and %eax, 0xc(%esp)
```

```
i=ilj;
```

```
0x401366 mov 0x8(%esp), %eax
```

```
0x40136a or %eax, 0xc(%esp)
```

```
i=~i;
```

```
0x40136e notl 0xc(%esp)
```

; 1 存入 0xc+esp所对应的地址单元 (i=1)

; -1 存入 0x8+esp所对应的地址单元 (j=-1)

; 0x8+esp地址单元内数据 (j) -> eax寄存器

; eax 与 0xc+esp存储单元数据 (i) 操作后 -> 0xc存储单元 (i)

3.1.2 C语言中的逻辑运算

- 逻辑运算属于非数值运算，所有参与运算的数以及运算结果只有0、1两个值。所有非0值都当做1处理。

- ① && 与
- ② || 或
- ③ ! 非

- C语言中逻辑运算和汇编代码的对应关系：

```
int i=2,j=-1;
```

```
0x40134e movl $0x2, 0xc(%esp)
```

```
0x401356 movl $0xffffffff, 0x8(%esp)
```

```
i=i&& j;
```

```
0x40135e cmpl $0x0, 0xc(%esp)
```

```
0x401363 je 0x401373<main+51>
```

```
0x401365 cmpl $0x0, 0x8(%esp)
```

```
0x40136a je 0x401373<main+51>
```

```
0x40136c mov $0x1, %eax
```

```
0x401371 jmp 0x401378<main+56>
```

```
0x401373 mov $0x0, %eax
```

```
0x401378 mov %eax, 0xc(%esp)
```

; 0xc+esp存储单元 (i) 与0进行比较

; 如果相等 (i=0) 则转0x401373 ; 51=33H

; 0x8+esp存储单元 (j) 与0进行比较

; 如果相等 (j=0) 则转0x401373

; 1 -> eax寄存器; 确认非0后, 结果置1

; 转0x401378 ; 0 -> eax寄存器

; eax -> 0xc+esp存储单元 (i)

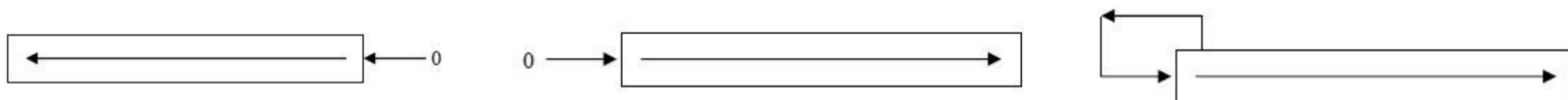
3.1.3 C语言中的移位运算

- ① << 左移，对应逻辑左移
- ② >> 右移，如果是无符号数则是逻辑右移，如果是有符号数则是算术右移

逻辑左移

逻辑右移

算术右移



- C语言中移位运算和汇编代码的对应关系：

```
int i=-1; unsigned j=2;
```

```
0x40134e movl $0ffffff, 0xc(%esp)
0x401356 movl $0x2, 0x8(%esp)
```

```
i=i<<7;
0x40135e shll $0x7, 0xc(%esp)
```

```
i=i>>8;
0x401363 sarl $0x8, 0xc(%esp)
```

```
j=j>>15;
0x401368 shrll $0xf, 0x8(%esp)
```

```
i=i>>j;
```

```
0x40136d mov 0x8(%esp), %eax
0x401371 mov %al, %cl
0x401373 sarl %cl, 0xc(%esp)
```

; -1 -> 0xc+esp对应的存储单元 (i=-1)

; 2 -> 0x8+esp对应的存储单元 (j=2)

; 0xc+esp存储单元 (i) 的内容**逻辑左移7位**

; 0xc+esp存储单元 (i) 的内容**算术右移8位**

; 0x8+esp存储单元 (j) 的内容逻辑右移15位

; 0x8+esp存储单元 (j) -> eax寄存器

; al寄存器 -> cl寄存器

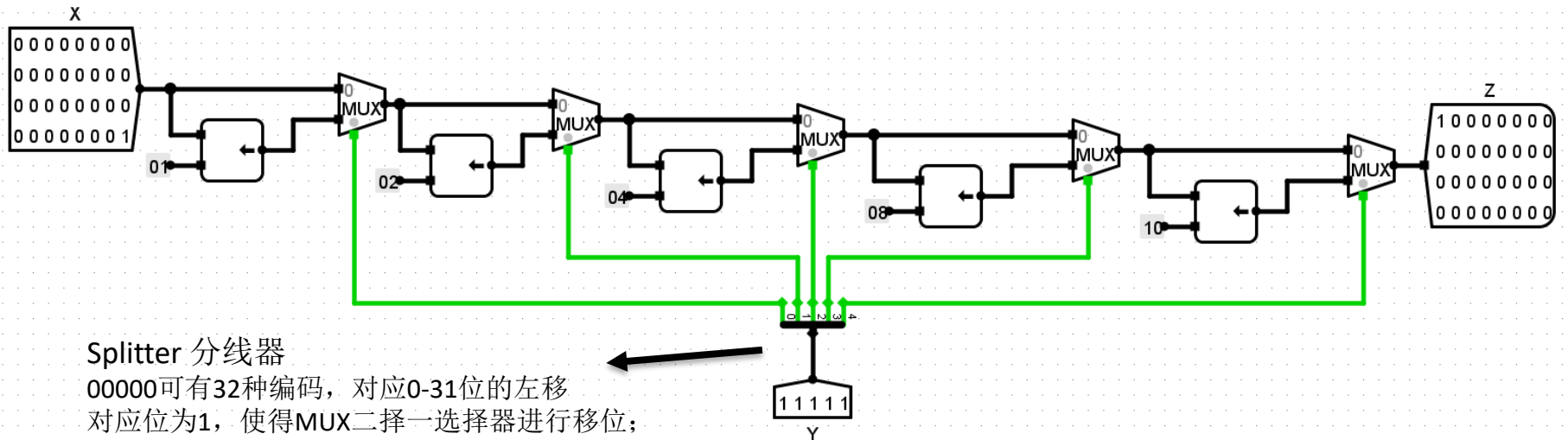
; 0xc+esp存储单元 (i) 的内容**算术右移cl位 (j位)**

3.1.3 C语言中的移位运算

- 左移：可能会产生**溢出**
 - `char x = 1111 1101 = -3` 逻辑左移 `x = 1111 1010 = -6` 不溢出
 - `unsigned char x = 1111 1101 = 253` 逻辑左移 `x = 11111010 = 250` （正确结果应该是 $253 \times 2 = 506$ 故**产生溢出**）
- 右移：可能会产生**误差**
 - `char x = 1111 1101 = -3` 算术右移 `x = 1111 1110 = -2` 有误差（正确结果应该是-1.5）
 - `unsigned char x = 1111 1101 = 253` 逻辑右移 `x = 0111 1110 = 126` 有误差（正确结果应该是126.5）
- 图3.1（见教材）：**桶型移位器(可变移位运算)**，实现 $x \ll y$ （左移）， x 为32位变量， y 为5位变量（ $y=0 \sim 31$ ）

Logisim实现的桶型移位器（左移0~31位）

设计文件“桶型移位器.circ”



输入：X(32位)，Y(5位)

输出：Z(32位)=X<<Y

桶型移位器

3.1.4 C语言中的算术运算

- ① + 加
- ② - 减
- ③ * 乘
- ④ / 除

- 汇编指令中有符号数和无符号数的加减运算是一样的：
 - **add**（加）、**sub**（减）
- 汇编指令中有符号数和无符号数的乘除运算是不一样的：
 - 有符号数：**mul**（乘）、**div**（除）
 - 无符号数：**imul**（乘）、**idiv**（除）

3.1.4 C语言中的算术运算

– C语言中算术运算和汇编代码的对应关系:

单字节无符号整数

```
int i,j; unsigned char ui, uj=255;
```

```
0x40134e movb $0ff, 0x1f(%esp)
```

; 255 -> 0x1f+esp为地址的存储单元 (uj=255)

```
float e=1,f;
```

```
0x401353 mov 0x404140, %eax
```

; 0x404140存储单元的内容 (=1) -> eax寄存器

```
0x401358 mov %eax, 0x18(%esp)
```

; eax -> 0x18+esp为地址的存储单元 (e=1)

```
i=i+3;
```

```
0x40135c addl $0x3, 0x14(%esp)
```

; 0x14+esp存储单元的内容 (i) + 3 -> 0x14+esp存储单元(i)

```
jui=ui-3;
```

```
0x401361 subb $0x3, 0x13(%esp)
```

; 0x13+esp存储单元的内容 (ui) - 3 -> 0x13+esp存储单元(ui)

```
i=i*j;
```

```
0x401366 mov 0x14(%esp), %eax
```

; 0x14+esp存储单元的内容 (i) -> eax寄存器

```
0x40136a imul 0xc(%esp), %eax
```

; 0xc+esp存储单元的内容 (j) * eax -> eax寄存器

```
0x40136f mov %eax, 0x14(%esp)
```

; eax寄存器 -> 0x14+esp存储单元 (i)

3.1.4 C语言中的算术运算

ui=ui*uj;

```
0x401373 mov 0x13(%esp),%al
0x401377 mulb 0x1f(%esp)
0x40137b mov %al,0x13(%esp)
```

i=i/j

```
0x40137f mov 0x14(%esp),%eax
0x401383 cld
0x401384 idivl 0xc(%esp)
0x401388 mov %eax, 0x14(%esp)
```

ui=ui/uj

```
0x40138c mov 0x13(%esp), %al
0x401390 movzbl %al,%eax
0x401393 divb 0x1f(%esp)
0x401397 mov %al,0x20(%esp)
```

e=e+f

单精度浮点数相加

```
0x40139b flds 0x18(%esp)
0x40139f fadds 0x20(%esp)
0x13a3 fstps 0x18(%esp)
```

i=i+3;

```
0x40135c addl $0x3, 0x14(%esp)
```

jui=ui-3;

```
0x401361 subb $0x3, 0x13(%esp)
```

; 0x13+esp存储单元的内容 (ui) -> al寄存器

; al寄存器*0x1f+esp存储单元的内容 (uj) -> al寄存器

; al寄存器 -> 0x13+esp存储单元 (ui)

; 长整型->双精度 32bit->64bit

; 0x14+esp存储单元的内容 (i) -> eax寄存器

; eax寄存器/0xc+esp存储单元的内容 (j) -> eax寄存器

; eax寄存器 -> 0x14+esp存储单元 (i)

; 0x13+esp存储单元的内容 (ui) -> al寄存器

; movzbl :将0扩展的1字节复制到4字节 al寄存器 -> eax寄存器

movzbl:将一个字节的数据拷贝到一个32位的寄存器中, 并在高位填充零。用于无符号数的扩展

; eax寄存器/0x1f+esp存储单元的内容 (uj) -> eax寄存器

; 将0x18+esp存储单元的内容 (e) -> 浮点寄存器

; 浮点寄存器+0x20+esp存储单元 (f) -> 浮点寄存器

; 浮点寄存器 -> 0x18+esp存储单元 (e) 存储

3.2 定点加减法运算

• 3.2.1 补码加减法运算方法

– 1、补码加法

- $[x+y]_{\text{补}} = [x]_{\text{补}} + [y]_{\text{补}} \pmod{M}$
- 例3.1: $x=0.1010$, $y=0.0101$, 求 $[x+y]_{\text{补}}$
- 解: $[x]_{\text{补}}=0.1010$, $[y]_{\text{补}}=0.0101$; $[x+y]_{\text{补}} = [x]_{\text{补}} + [y]_{\text{补}}$

$$\begin{array}{r} [x]_{\text{补}} \quad 0.1010 \\ + [y]_{\text{补}} \quad 0.0101 \\ \hline [x+y]_{\text{补}} \quad 0.1111 \end{array}$$

- $[x+y]_{\text{补}}=0.1111$

3.2.1 补码加减法运算方法

- 例3.2: $x=-0.1010$, $y=-0.0100$, 求 $[x+y]_{\text{补}}$ 和 $x+y$

- 解: $[x]_{\text{补}}=1.0110$, $[y]_{\text{补}}=1.1100$; $[x+y]_{\text{补}}=[x]_{\text{补}}+[y]_{\text{补}}$

$$\begin{array}{r} [x]_{\text{补}} \quad 1.0110 \\ + [y]_{\text{补}} \quad 1.1100 \\ \hline [x+y]_{\text{补}} \quad 11.0010 \end{array}$$

最高位1去掉

- $[x+y]_{\text{补}}=1.0010$ $x+y=-0.1110$

取反加1

1010取反得到0101, 加1得到0110

0100取反得到1011, 加1得到1100

取反加1

0010取反得到1101, 加1得到1110

3.2.1 补码加减法运算方法

— 2、补码减法

$[-Y]_{\text{补}} = [Y]_{\text{补}}$ 连同符号位取反+1

• $[x-y]_{\text{补}} = [x]_{\text{补}} - [y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}} \pmod{M}$

• 例3.3: $x=0.1001$, $y=0.0110$, 求 $[x-y]_{\text{补}}$

• 解: $[x]_{\text{补}}=0.1001$, $[y]_{\text{补}}=0.0110$, $[-y]_{\text{补}}=1.1010$; $[x-y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$

$$\begin{array}{r} [x]_{\text{补}} \quad 0.1001 \\ + [-y]_{\text{补}} \quad 1.1010 \\ \hline [x-y]_{\text{补}} \quad 10.0011 \end{array}$$

最高位1去掉

模溢出

取反加1

0110取反得到1001, 加1得到1010

取反加1

1001取反得到0110, 加1得到0111
0110取反得到1001, 加1得到1010

取反加1

1101取反得到0010, 加1得到0011

• $[x-y]_{\text{补}} = 0.0011$

$$\begin{array}{r} [x]_{\text{补}} \quad 1.0111 \\ + [-y]_{\text{补}} \quad 0.0110 \\ \hline [x-y]_{\text{补}} \quad 1.1101 \end{array}$$

• 例3.4: $x=-0.1001$, $y=-0.0110$, 求 $[x-y]_{\text{补}}$ 和 $x-y$

• 解: $[x]_{\text{补}}=1.0111$, $[y]_{\text{补}}=1.1010$, $[-y]_{\text{补}}=0.0110$; $[x-y]_{\text{补}} = [x]_{\text{补}} + [-y]_{\text{补}}$

• $[x-y]_{\text{补}}=1.1101$ $x-y = -0.0011$

3.2.2 溢出及检测

– 1、溢出的概念

- 例3.5: (1) 设 $[x]_{\text{补}}=0.1011$, $[y]_{\text{补}}=0.1100$, 求 $[x+y]_{\text{补}}$
(2) 设 $[x]_{\text{补}}=1.0101$, $[y]_{\text{补}}=1.0100$, 求 $[x+y]_{\text{补}}$

• 解:

$$\begin{array}{r} [x]_{\text{补}} \quad 0.1011 \\ + [y]_{\text{补}} \quad 0.1100 \\ \hline \end{array}$$

$$[x+y]_{\text{补}} \quad 1.0111$$

$$\begin{array}{r} [x]_{\text{补}} \quad 1.0101 \\ + [y]_{\text{补}} \quad 1.0100 \\ \hline \end{array}$$

$$[x+y]_{\text{补}} \quad 10.1001$$

最高位1去掉

- 第(1)种情况: 两个正数相加, 结果为负数(1.0111), 出错
- 第(2)种情况: 两个负数相加, 结果为正数(0.1001), 出错
- 原因:
 - 第(1)种情况, $x+y=11/16+12/16=23/16$ 的值大于1, 超出了定点小数的表示范围, 发生溢出
 - 第(2)种情况, $x+y=(-11/16)+(-12/16)=(-23/16)$ 的值小于-1, 也超出了定点小数的表示范围, 发生溢出
 - 补码定点小数表示范围: $[-1,1)$

3.2.2 溢出及检测

– 2、溢出检测

三种常见的溢出检测方法：

- (1) 正数 + 正数 = 正数 不溢出
- 正数 + 正数 = 负数 溢出
 - 负数 + 负数 = 负数 不溢出
 - 负数 + 负数 = 正数 溢出
 - 正数 + 负数 不可能产生溢出
 - 负数 + 正数 不可能产生溢出
 - 根据操作数和运算结果的符号位是否一致进行检测
 - 正正得负、负负得正时溢出
 - 设 x_f 、 y_f 为操作数的符号位， s_f 为运算结果的符号位
 - 溢出标志 v 检测公式：
$$v = x_f \overline{y_f} \overline{s_f} + \overline{x_f} y_f \overline{s_f}$$
 - $v=0$ ，没有溢出； $v=1$ ，有溢出
 - 如果是减法，则溢出标志 v 的检测公式为：
$$v = x_f \overline{y_f} \overline{s_f} + \overline{x_f} y_f s_f$$

– $[x-y]_{\text{补}} = [x]_{\text{补}} - [y]_{\text{补}}$

3.2.2 溢出及检测

- 例3.6: $x=-0.1011$, $y=0.1100$, 求 $[x-y]_{\text{补}}$
- 解: $[x]_{\text{补}}=1.0101$, $[y]_{\text{补}}=0.1100$, $[-y]_{\text{补}}=1.0100$

$$\begin{array}{r} [x]_{\text{补}} \quad 1.0101 \\ + [-y]_{\text{补}} \quad 1.0100 \\ \hline [x-y]_{\text{补}} \quad 10.1001 \end{array}$$

最高有效位向前无进位
但符号位向前有进位
所以溢出

- 负数-正数=负数+负数, 结果是正数, 溢出了

- 溢出标志 V : $V = X_f Y_f \bar{S}_f + \bar{X}_f \bar{Y}_f S_f$

- $X_f=1$, $Y_f=1$ ($[-y]_{\text{补}}$ 的符号位), $S_f=0$
- $V = 1 \cdot 1 \cdot 1 + 0 \cdot 0 \cdot 0 = 1$, 溢出

- 如果采用减法运算的溢出标志 V : $V = X_f \bar{Y}_f \bar{S}_f + \bar{X}_f Y_f S_f$

$$- [x-y]_{\text{补}} = [x]_{\text{补}} - [y]_{\text{补}}$$

- $X_f=1$, $Y_f=0$ ($[y]_{\text{补}}$ 的符号位), $S_f=0$
- $V = 1 \cdot 1 \cdot 1 + 0 \cdot 0 \cdot 0 = 1$, 溢出

3.2.2 溢出及检测

— (2) 根据运算过程中最高数据位的进位与符号位的进位是否一致进行检测

- 溢出标志V检测公式:

$$V = C_f \oplus C_d$$

- C_f : 符号位产生的进位信号; C_d : 最高有效位产生的进位信号

- 例3.7: $x=-111$, $y=110$, 求 $[x-y]_{\text{补}}$

- 解: $[x]_{\text{补}}=1,001$, $[y]_{\text{补}}=0,110$, $[-y]_{\text{补}}=1,010$

- $C_f = 1$; $C_d = 0$

- 溢出标志 $V=1\oplus 0=1$, 溢出 ($-7-6=-13$; 超出 $-8\sim+7$ 的范围)

- 例3.8: $x=-011$, $y=100$, 求 $[x-y]_{\text{补}}$

- 解: $[x]_{\text{补}}=1,101$, $[y]_{\text{补}}=0,100$, $[-y]_{\text{补}}=1,110$

- $C_f = 1$; $C_d = 1$

- 溢出标志 $V=1\oplus 1=0$, 没有溢出 ($-3-4=-7$; 没有超出 $-8\sim+7$ 的范围)

$[x]_{\text{补}}$	1,001
$+ [-y]_{\text{补}}$	1,010
$[x-y]_{\text{补}}$	10,011

$C_f = 1$ $C_d = 0$

$[x]_{\text{补}}$	1,101
$+ [-y]_{\text{补}}$	1,110
$[x-y]_{\text{补}}$	11,001

$C_f = 1$ $C_d = 1$

最高有效位向前有进位
符号位向前也有进位
所以无溢出

3.2.2 溢出及检测

(3) 利用变形补码的符号位进行检测

- 变形补码（双符号位补码）：正数的符号位为00，负数的符号位为11
- 如果运算结果的符号位为00或11，则没有溢出；如果运算结果的符号位为01或10，则产生溢出
- 溢出标志V检测公式： $V = S_{f1} \oplus S_{f2}$
- S_{f1} 、 S_{f2} 分别表示运算结果的第1个符号位和第2个符号位
- Sf1为结果真正符号位

例3.9:

- (1) 设 $[x]_{\text{补}} = 00.1011$, $[y]_{\text{补}} = 00.0111$, 求 $[x+y]_{\text{补}}$
- (2) 设 $[x]_{\text{补}} = 11.0101$, $[y]_{\text{补}} = 11.0011$, 求 $[x+y]_{\text{补}}$
- (3) 设 $[x]_{\text{补}} = 00.1011$, $[y]_{\text{补}} = 11.0011$, 求 $[x+y]_{\text{补}}$

解:

- 第(1)情况，运算结果的符号位为01，正溢出 ($11/16 + 7/16 = 18/16$)
- 第(2)情况，运算结果的符号位为10，负溢出 ($-11/16 - 13/16 = -24/16$)
- 第(3)情况，运算结果的符号位为11，没有溢出 ($11/16 - 13/16 = -2/16$)

$$\begin{array}{r} [x]_{\text{补}} \quad 00.1011 \\ + [y]_{\text{补}} \quad 00.0111 \\ \hline [x+y]_{\text{补}} \quad 01.0010 \end{array}$$

(1)

$$\begin{array}{r} [x]_{\text{补}} \quad 11.0101 \\ + [y]_{\text{补}} \quad 11.0011 \\ \hline [x+y]_{\text{补}} \quad 10.1000 \end{array}$$

(2)

$$\begin{array}{r} [x]_{\text{补}} \quad 00.1011 \\ + [y]_{\text{补}} \quad 11.0011 \\ \hline [x+y]_{\text{补}} \quad 11.1110 \end{array}$$

(3)

3.2.2 溢出及检测

- 溢出检测的软件方法:

```
int tadd_ok(int x,int y) {  
    int sum=x+y;  
    int neg_over=x<0&&y<0&&sum>=0;  
    int pos_over=x>=0&&y>=0&&sum<0;  
    return !neg_over&&!pos_over;  
}
```

负数 + 负数 = 正数	溢出 (负溢出)
--------------	----------

正数 + 正数 = 负数	溢出 (正溢出)
--------------	----------

3.2.3 加减法的逻辑实现

– 1、全加器（Full Adder, FA）

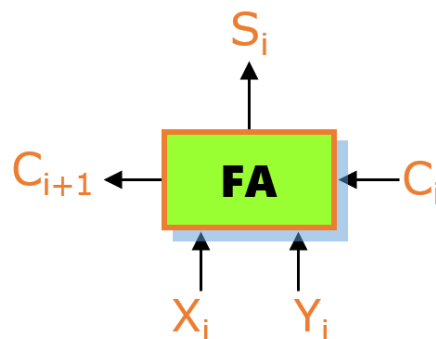
加数 X_i	加数 Y_i	低位进位 C_i	和数 S_i	进位 C_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$S_i = X_i \oplus Y_i \oplus C_i$$

$$C_{i+1} = X_i Y_i + (X_i \oplus Y_i) C_i$$

或者

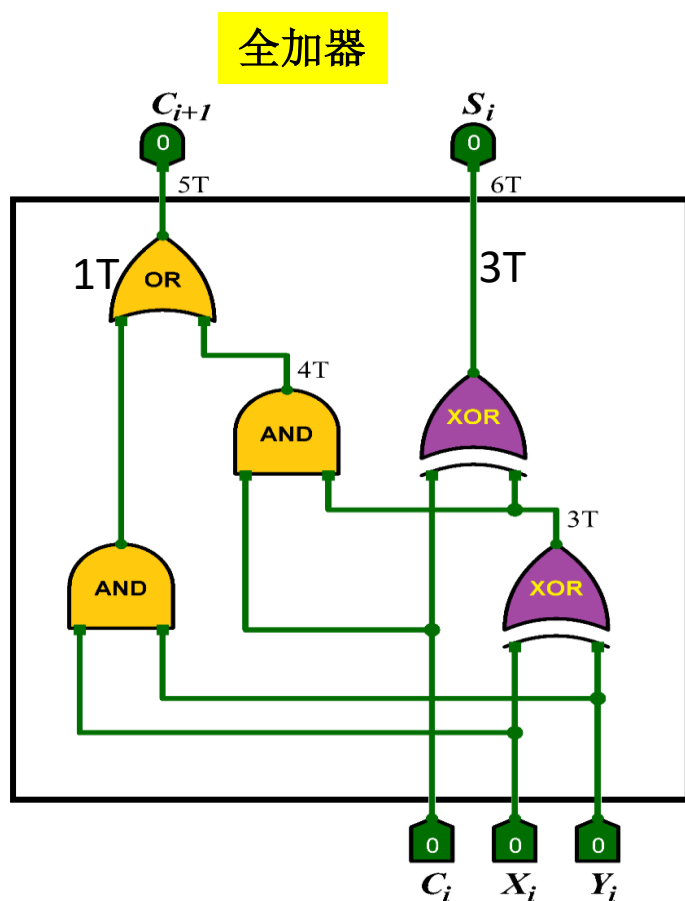
$$C_{i+1} = X_i Y_i + (X_i + Y_i) C_i$$



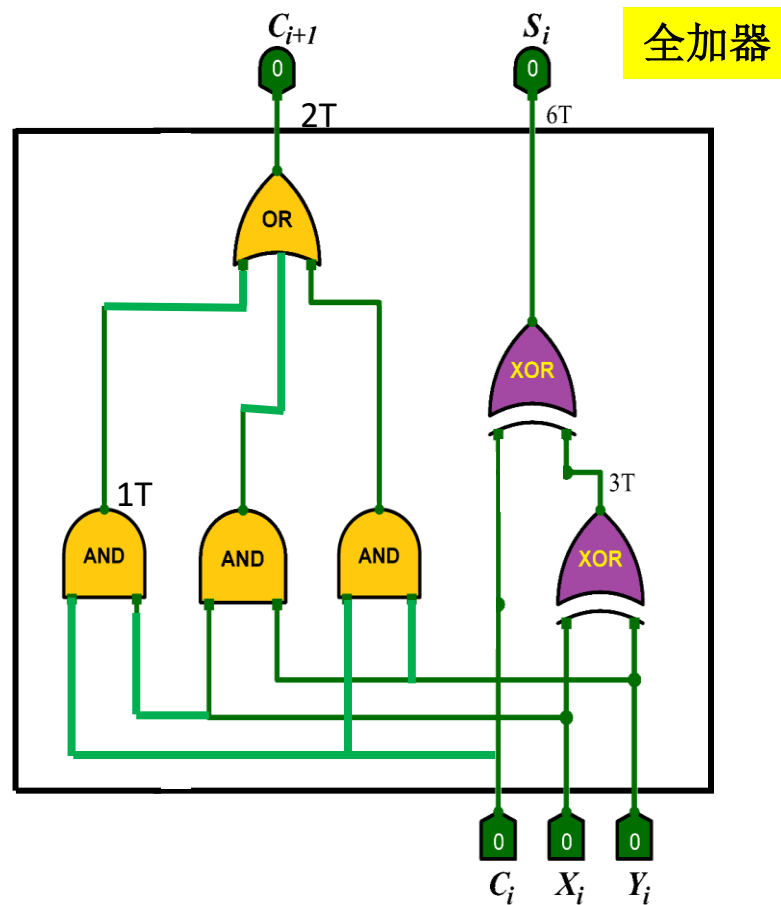
3.2.3 加减法的逻辑实现

— 全加器逻辑电路实现：

- 假设异或门的时间延迟为 $3T$ 、与门和或门的时间延迟均为 $1T$



$$C_{i+1} = X_i Y_i + (X_i \oplus Y_i) C_i$$

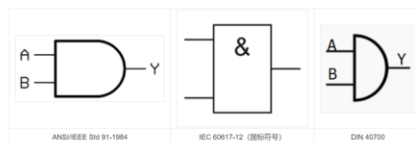


或者

$$C_{i+1} = X_i Y_i + (X_i + Y_i) C_i$$

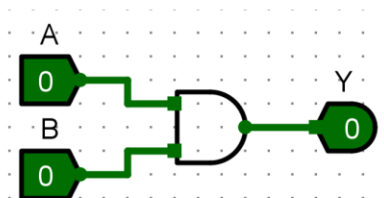
逻辑门（1）

• 与门： $Y=A \cdot B$

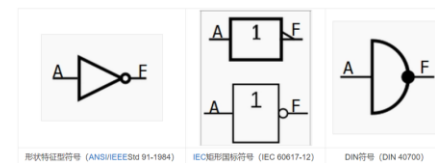


真值表

输入A	输入B	输出Y
0	0	0
0	1	0
1	0	0
1	1	1

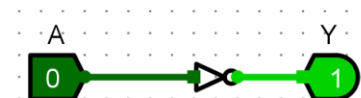


• 非门： $Y=\overline{A}$



真值表

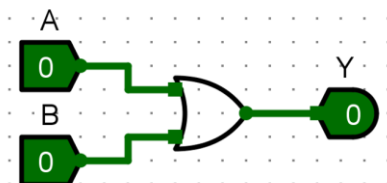
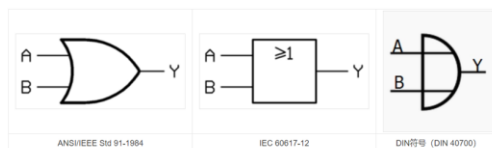
输入A	输出Y
0	1
1	0



• 或门： $Y=A+B$

真值表

输入A	输入B	输出F
0	0	0
0	1	1
1	0	1
1	1	1

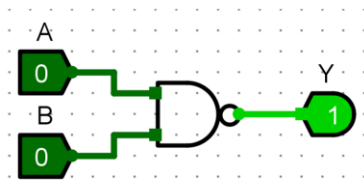
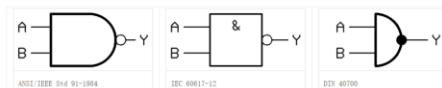


逻辑门 (2)

• 与非门: $Y = \overline{A \cdot B} = \overline{A} + \overline{B}$

真值表

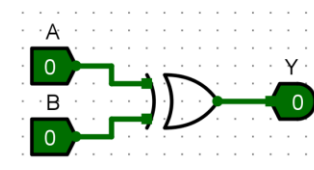
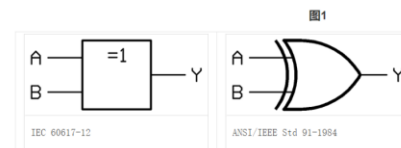
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0



• 异或门: $Y = A \oplus B = \overline{A} \cdot B + A \cdot \overline{B}$

真值表

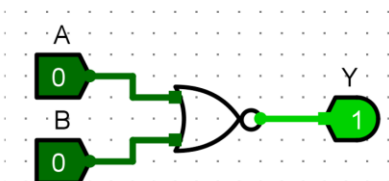
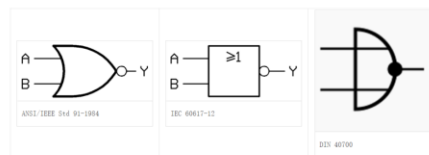
A	B	输出Y
0	0	0
0	1	1
1	0	1
1	1	0



• 或非门: $Y = \overline{A + B} = \overline{A} \cdot \overline{B}$

真值表

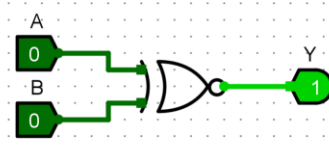
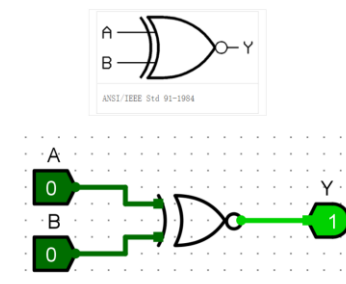
输入A	输入B	输出Y
0	0	1
0	1	0
1	0	0
1	1	0



• 同或门 (异或非门): $Y = \overline{A \oplus B} = A \cdot B + \overline{A} \cdot \overline{B}$

真值表

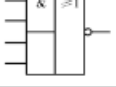
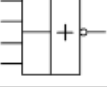
输入A	输入B	输出F
0	0	1
0	1	0
1	0	0
1	1	1



附录 1 基本逻辑门电路图形符号

表 C1 列出了基本逻辑门的国际图形符号和限定符号 (GB/T 4728.12-1996)、国外流行图形符号和曾用图形符号。

表 C1 基本逻辑门电路图形符号

序号	名称	GB/T 4728.12-1996		国外流行图形符号	曾用图形符号
		限定符号	国标图形符号		
1	与门	&			
2	或门	≥ 1			
3	非门	 逻辑非入和出	 	 	 
4	与非门				
5	或非门				
6	与或非门				
7	异或门	$\neq 1$			
8	同或门	=	 		 
9	集电极开路 OC 门、漏极 开路 OD 门	 L 型开路输出			
10	缓冲器	$\triangleright$			

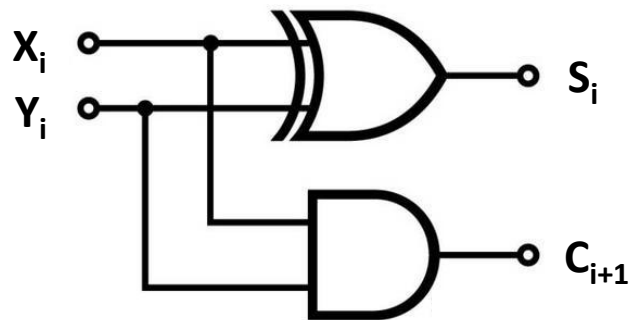
3.2.3 加减法的逻辑实现

— 半加器 (Half Adder, HA)

- $S_i = X_i \oplus Y_i$

- $C_{i+1} = X_i \cdot Y_i$

X_i	Y_i	S_i	C_{i+1}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



3.2.3 加减法的逻辑实现

$$C_{i+1} = X_i Y_i + (X_i \oplus Y_i) C_i$$

$$S_i = X_i \oplus Y_i \oplus C_i$$

– 2、多位串行加法器

最高数据位的进位 (C_{n-1}) 与符号位的进位 (C_n) 是否一致

- 也称为行波进位加法器（Ripple Carry Adder）

参考全加器电
路图3.2(a)
异或门3T
与门/或门 1T

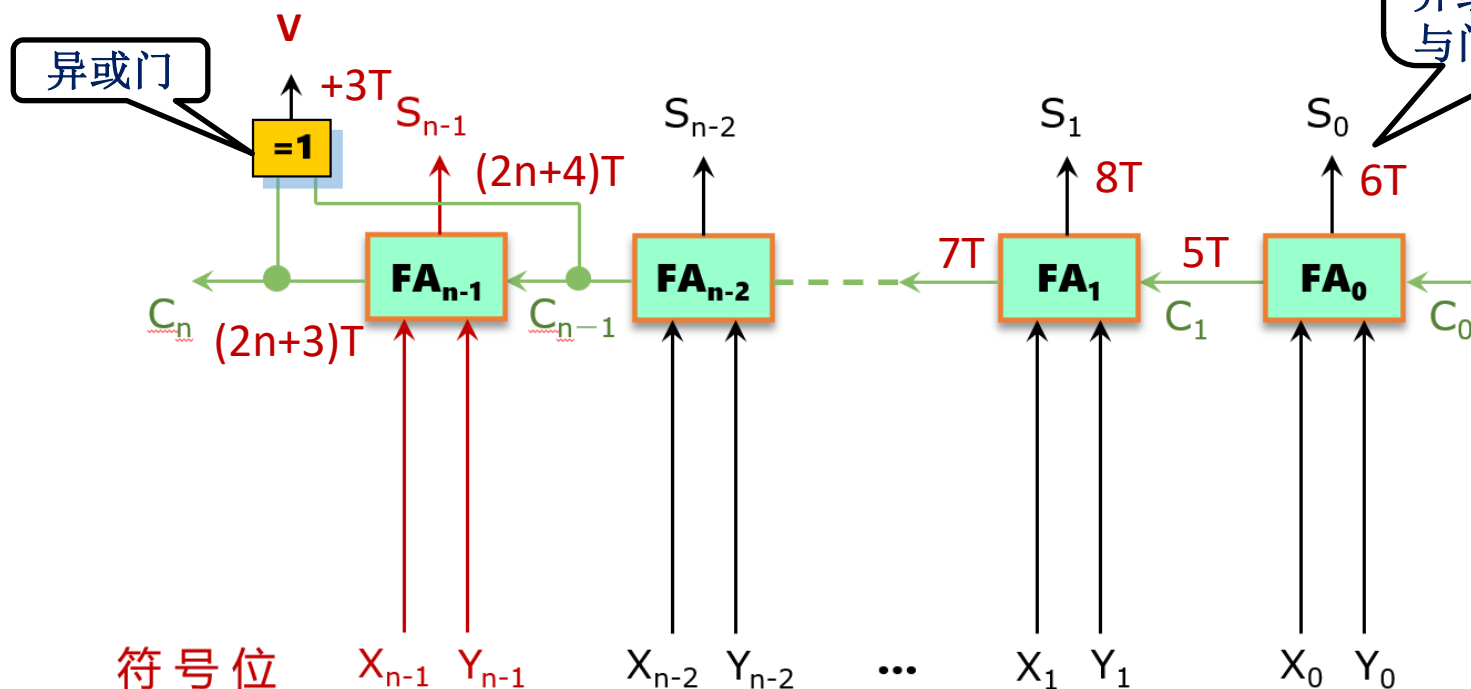
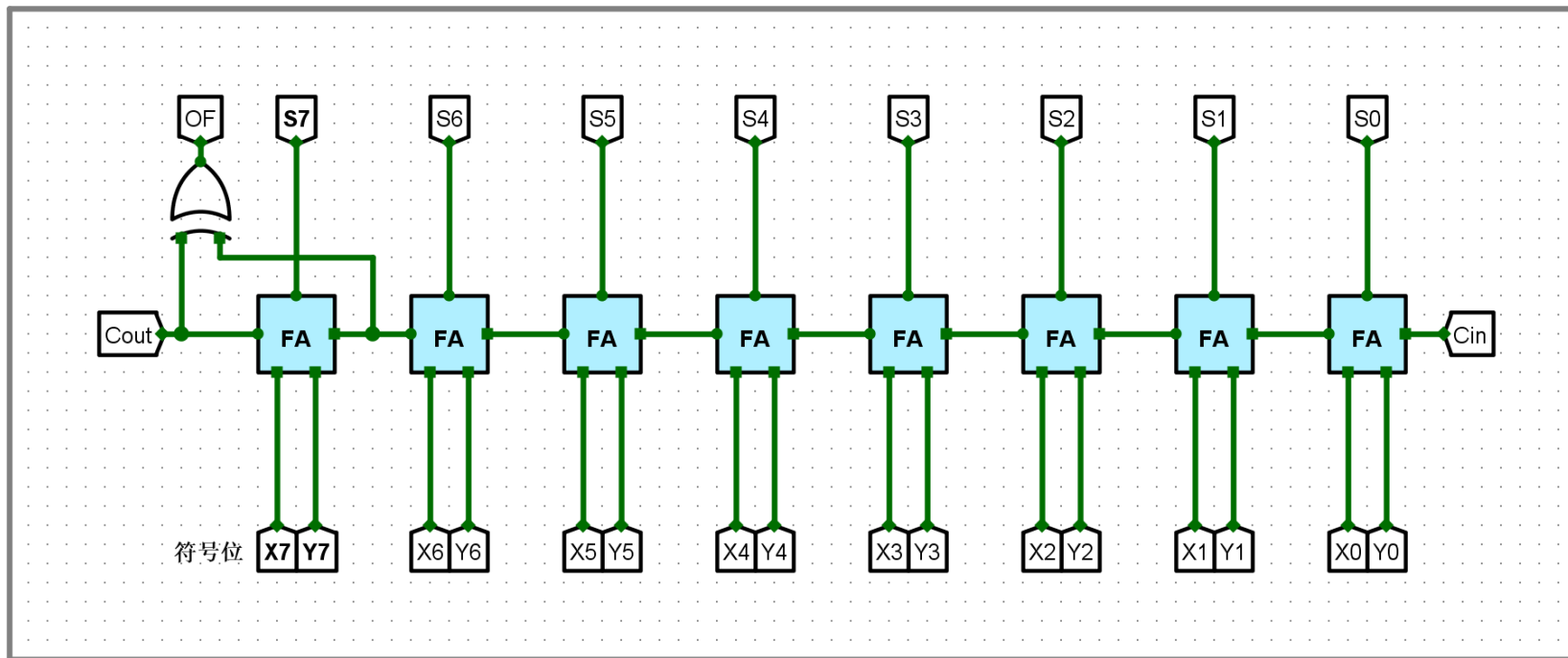


图3.3 多位串行加法器

实验：Logisim实现的8位串行加法器

设计文件“8位串行加法器.circ”



8位串行加法器

3.2.3 加减法的逻辑实现

– 3、可控加减法电路（Controller Adder / Subtractor）

$/Y$: Y 的非

- Sub=0: 加法运算
- Sub=1: 减法运算 ($X-Y=X+Y+1$, Sub=1, $Y \oplus \text{Sub} = /Y$, $C_0 = \text{Sub} = 1$)
 - Y_i 各位 $\oplus$ sub =1相当于取反, 即: $Y_i=0 \oplus \text{sub} \Rightarrow 1$; $Y_i=1 \oplus \text{sub} \Rightarrow 0$
 - 相当取反码, 结果+1, 体现在 C_0 , sub $\rightarrow C_0$.
- 无符号加法: 溢出标志 $V=C_n$; 无符号减法: 溢出标志 $V=/C_n$
- 有符号加法/减法: 溢出标志 $V=C_n \oplus C_{n-1}$

补码=原码取反+1

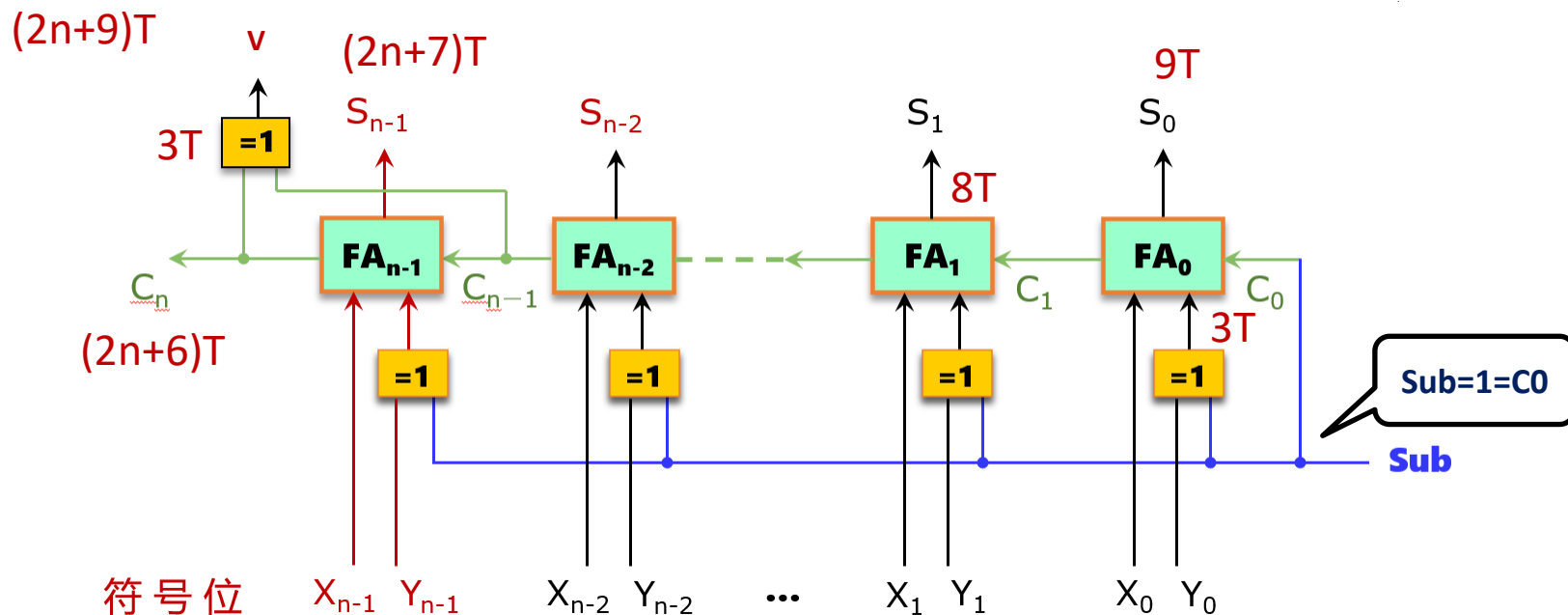


图3.4 多位可控加减法电路

3.2.3 加减法的逻辑实现

加法:
Cn=0 V=0
不溢出

- 无符号加法: 溢出标志 $V=C_n$
- 无符号减法: 溢出标志 $V=\neg C_n$

加法:
Cn=1 V=1
溢出

3+8=11

0	0	0	0	0	1	0	1
+	0	0	0	0	1	0	0
0	0	0	0	1	1	0	1

加法不溢出

255+254=253

1	1	1	1	1	1	1	1
+	1	1	1	1	1	1	0
1	1	1	1	1	1	0	1

加法溢出

128-1=127

1	0	0	0	0	0	0	0
+	1	1	1	1	1	1	1
1	0	1	1	1	1	1	1

减法不溢出

1-2=255

0	0	0	0	0	0	0	1
+	1	1	1	1	1	1	0
1	1	1	1	1	1	1	1

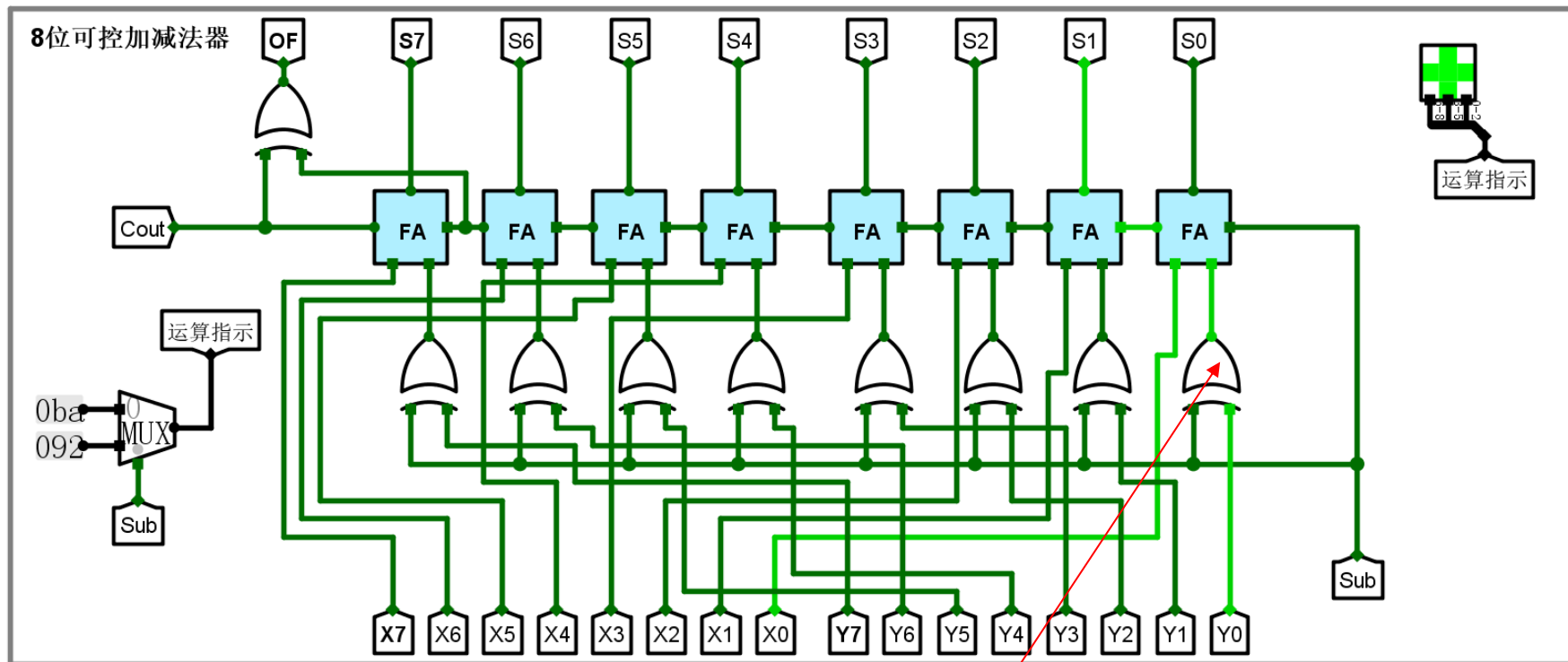
减法溢出

减法:
Cn=1 V=0
不溢出

减法:
Cn=0 V=1
溢出

实验：Logisim实现的8位串行可控加减法器

设计文件“8位串行可控加减法器.circ”



Sub=0: 加法运算

Sub=1: 减法运算 ($X-Y=X+Y+1$, Sub=1, $Y \oplus \text{Sub} = \neg Y$, $C_0 = \text{Sub} = 1$)

3.2.3 加减法的逻辑实现

– 4、先行进位加法器（Carry Look-Ahead Adder）

- 全加器：

- $S_i = X_i \oplus Y_i \oplus C_i$

- $C_{i+1} = X_i Y_i + (X_i \oplus Y_i) C_i$

- 定义：

- 进位生成函数： $G_i = X_i Y_i$

- 进位传递函数： $P_i = X_i \oplus Y_i$

- 则有：

- $C_{i+1} = G_i + P_i C_i$

- $C_1 = G_0 + P_0 C_0$

- $C_2 = G_1 + P_1 C_1 = G_1 + P_1 (G_0 + P_0 C_0) = G_1 + P_1 G_0 + P_1 P_0 C_0$

- $C_3 = G_2 + P_2 C_2 = G_2 + P_2 (G_1 + P_1 G_0 + P_1 P_0 C_0) = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$

- $C_4 = G_3 + P_3 C_3 = G_3 + P_3 (G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0) =$
 $G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$

- $C_n = G_{n-1} + P_{n-1} G_{n-2} + P_{n-1} P_{n-2} G_{n-3} + \dots + P_{n-1} P_{n-2} \dots P_1 P_0 C_0$

先行进位加法器

- 进位生成/进位传递电路（即教材中图3.6的与门异或门电路，时间延迟为 $3T$ ）：

进位生成函数： $G_i = X_i Y_i$

进位传递函数： $P_i = X_i \oplus Y_i$

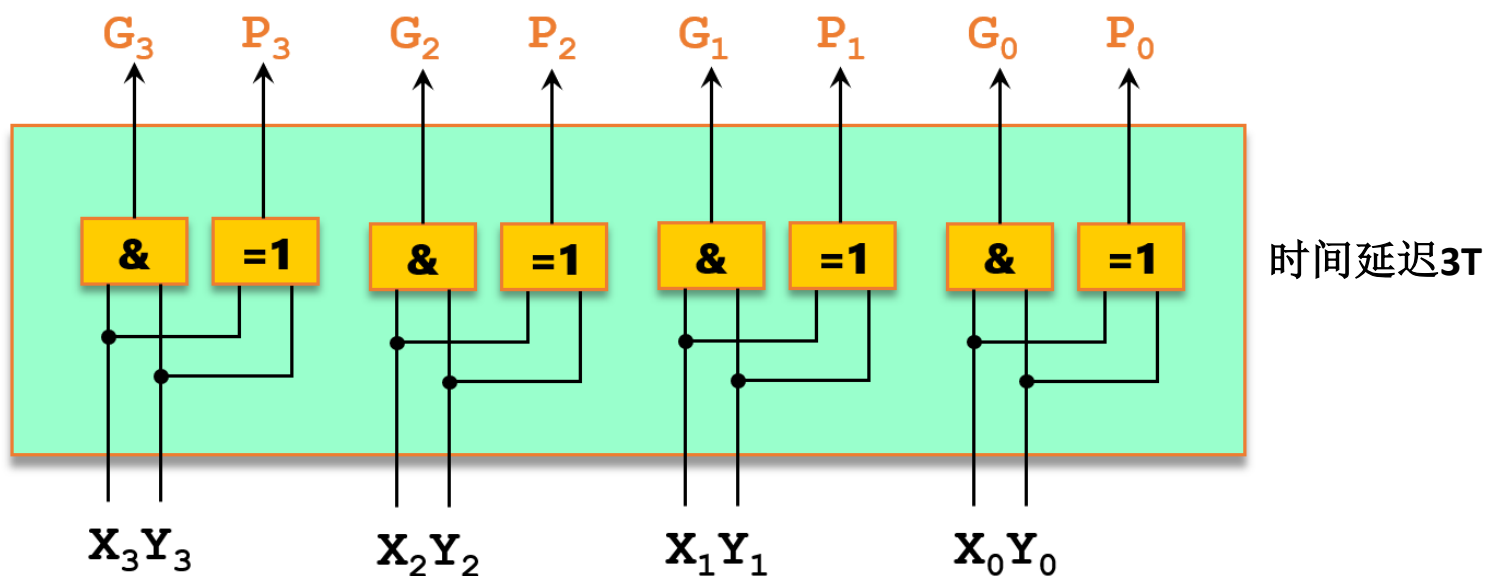


图3.6 4位进位生成/进位传递电路

先行进位加法器

- 4位先行进位电路（教材图3.5，时间延迟为2T）：

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 C_1 = G_1 + P_1(G_0 + P_0 C_0) = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 C_2 = G_2 + P_2(G_1 + P_1 G_0 + P_1 P_0 C_0) = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

$$C_4 = G_3 + P_3 C_3 = G_3 + P_3(G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0) = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

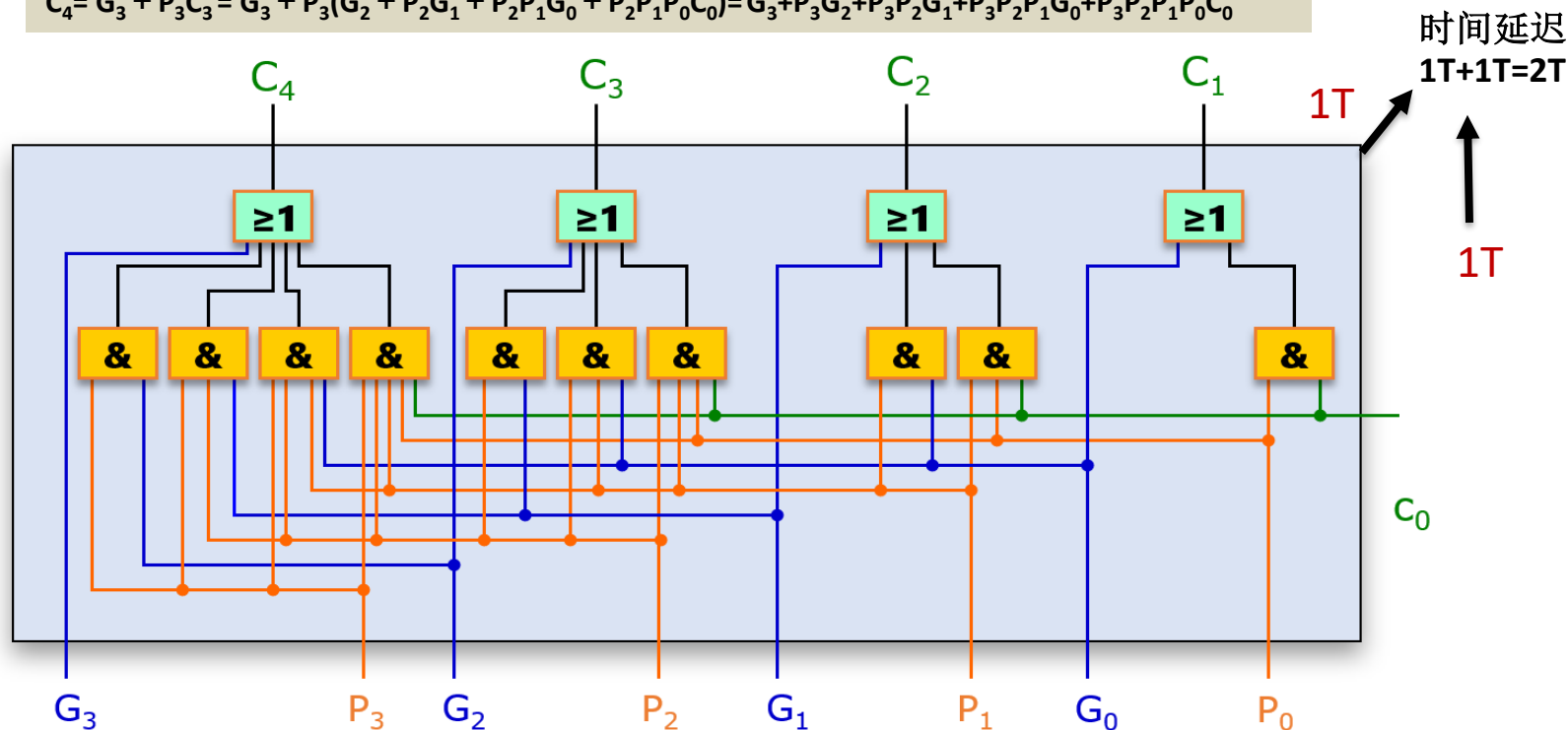


图3.5 4位先行进位电路

先行进位加法器

- 成组进位生成函数： G^*
- 成组进位传递函数： P^*
- 由前面的推导有：
 - $C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$
- 定义：
 - $G^* = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0$
 - $P^* = P_3 P_2 P_1 P_0$
- 则有：
 - $C_4 = G^* + P^* C_0$
- 设计成组进位生成和传递函数的目的：方便级联构成多级先行进位系统
SN74182

先行进位加法器

- 可级联的4位先行进位电路（CLA 74182，教材图3.5）：
 - 时间延迟为 $2T$
 - CLA 74182：也称为SN74182、74LS182，是4位先行进位电路芯片的名称

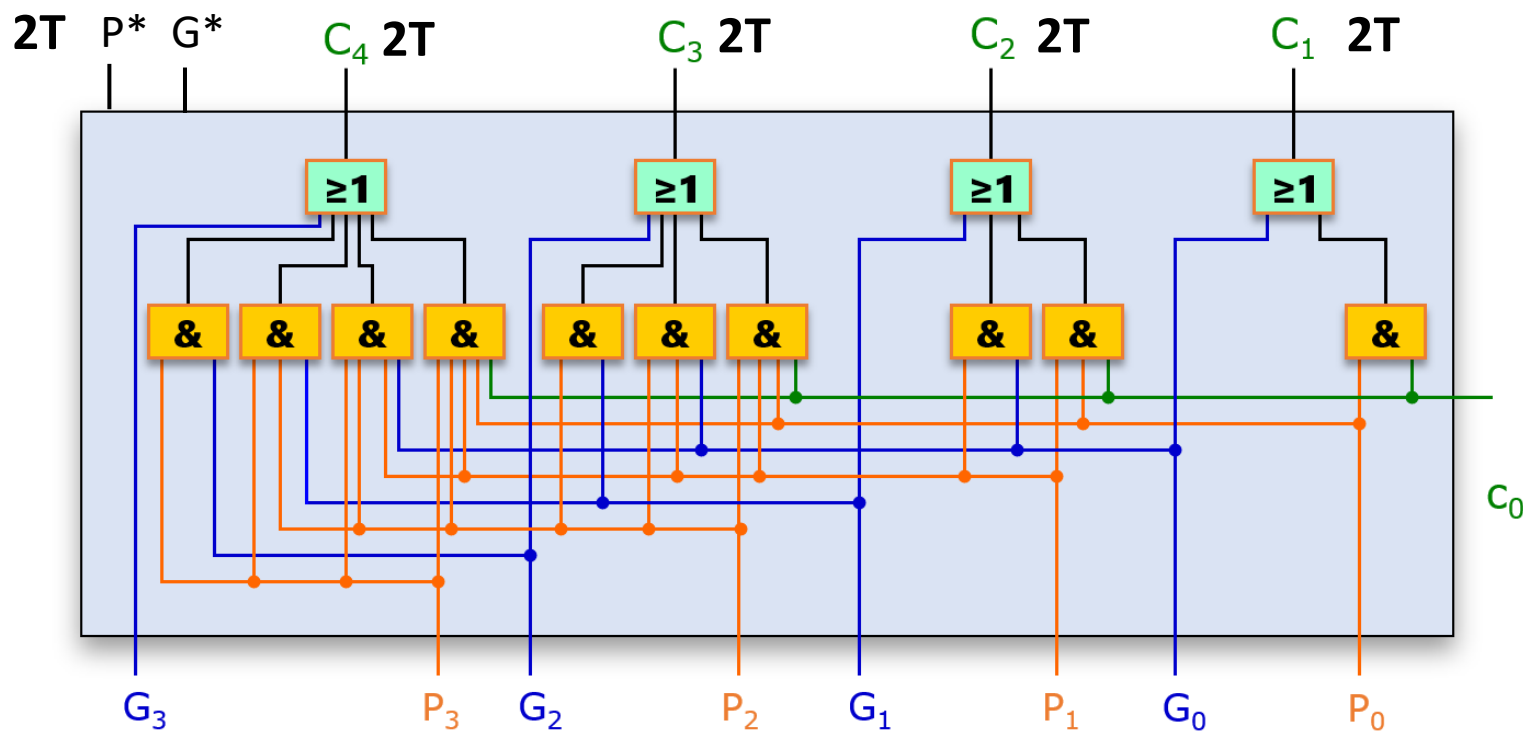


图3.5 可级联的4位先行进位电路

- 4位**先行**进位加法器（教材图3.6，也称快速加法器、并行加法器）
 - 得到 S_0 的时间延迟为 $6T$ （根据教材），得到 S_1 、 S_2 、 S_3 的时间延迟为 $3T+2T+3T=8T$
 - $S_0=P_0 \oplus C_0$ ，生成 P_0 的时间为 $3T$ ，再加上一个异或门的 $3T$ ，即为 $6T$
 - S_1, S_2, S_3 的时间要再加上 $2T$ 的进位电路传送时间，故为 $8T$
 - 得到 C_4 的时间延迟为 $2T+3T=5T$ ； $C_4=G_3+P_3G_2+P_3P_2G_1+P_3P_2P_1G_0+P_3P_2P_1P_0C_0$ （参图3.5）
 - X_i 和 Y_i 到位后，所有的 G 和 P 并行生成， C_0 到位后， C_1-C_4 同步， S_1-S_3 同步生成
 - 改之前的串行进位方式为并行进位

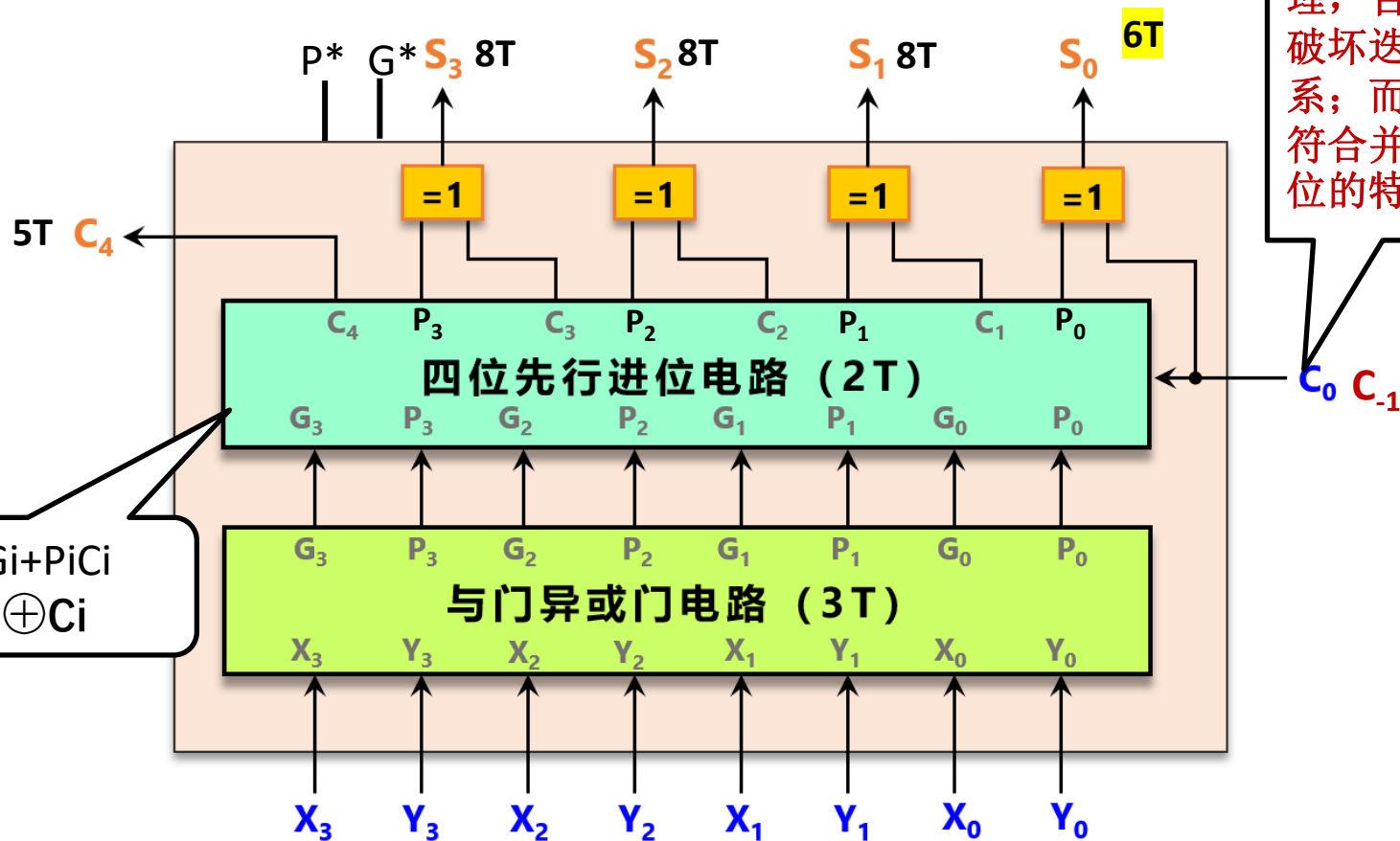
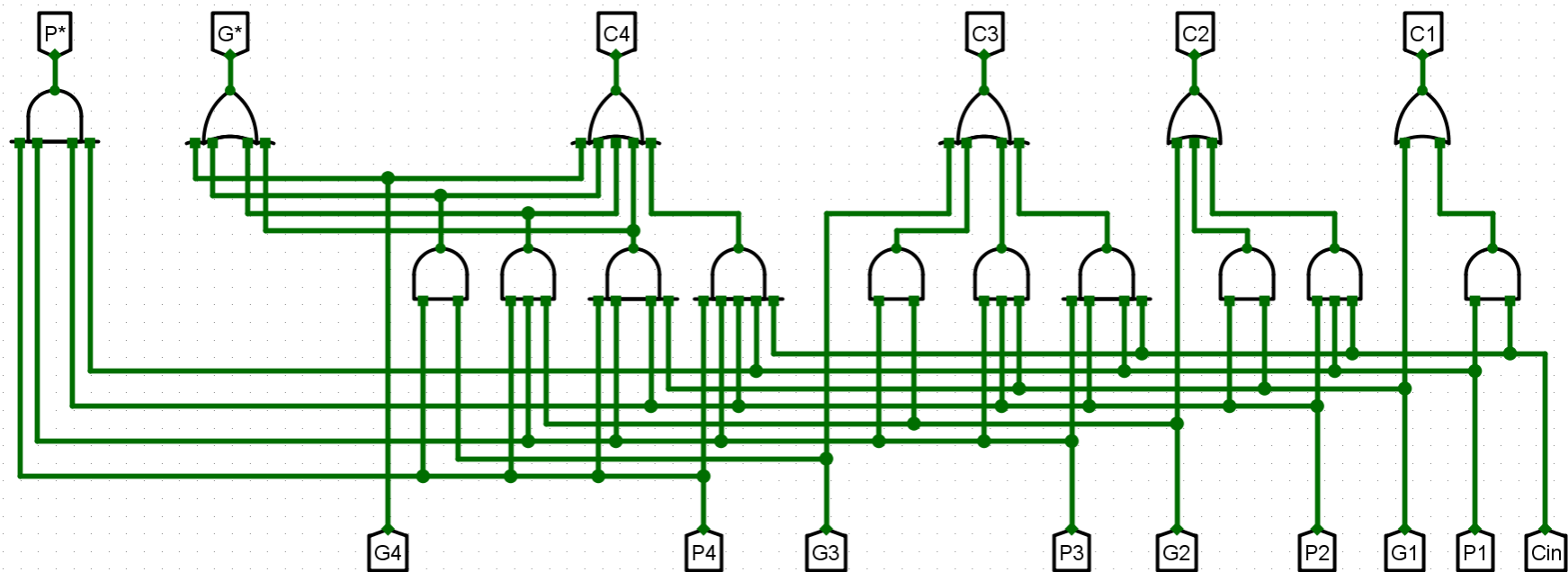


图3.6 4位先行进位加法器

Logisim实现的4位先行进位电路

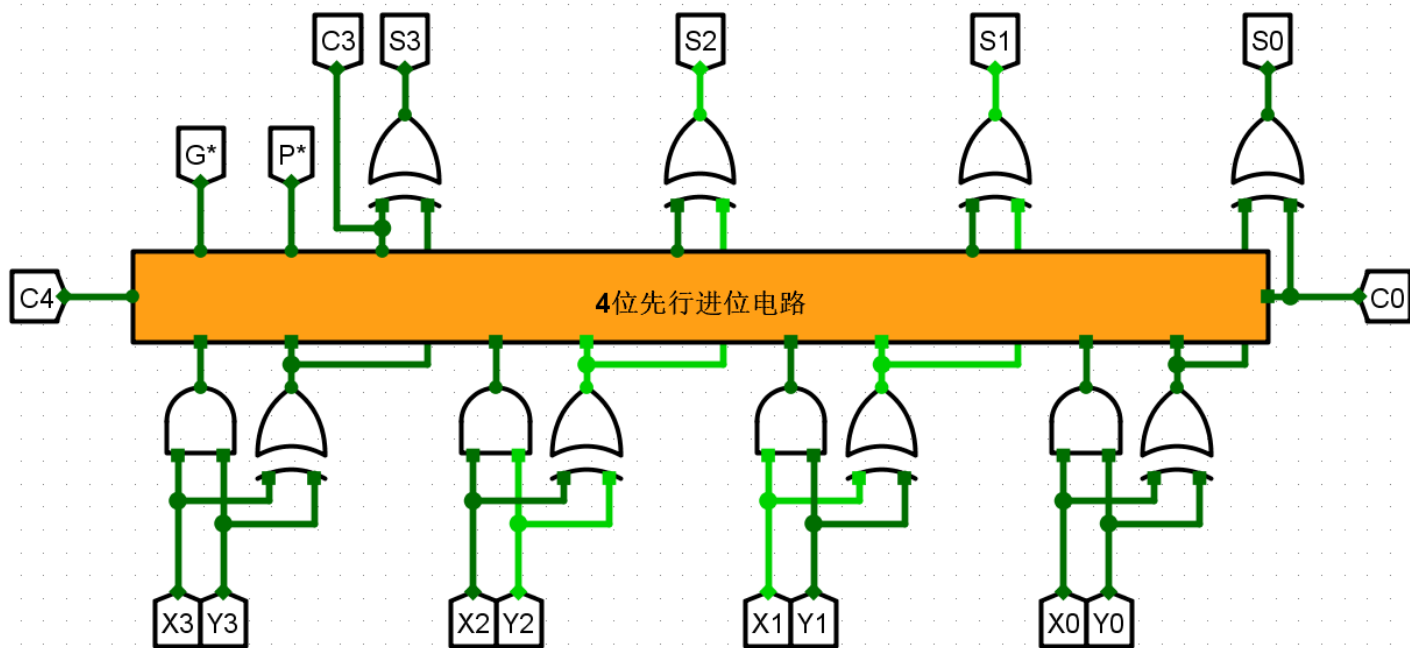
设计文件“4位先行进位电路.circ”



4位先行进位电路

Logisim实现的4位快速加法器（先行进位加法器）

设计文件“4位快速加法器.circ”



4位快速加法器

- 16位加法器（教材图3.7，组内并行、组间串行，即：单重分组）参：唐朔飞老师PPT
 - 得到 S_0 时间 $6T$ 、 S_1 、 S_2 、 S_3 的时间延迟为 $8T$ ； C_0 即刻到位，生成 S 参图3.6
 - $S_0 = X_0 \oplus Y_0 \oplus C_0$ ； $S_i = X_i \oplus Y_i \oplus C_i$ $C_{i+1} = X_i Y_i + (X_i \oplus Y_i) C_i$
 - 得到 S_4 、 S_5 、 S_6 、 S_7 的时间延迟为 $5T+5T=10T$ ；需要加上进位的生成时间 $5T$ 才能生成 S
 - 得到 S_8 、 S_9 、 S_{10} 、 S_{11} 的时间延迟为 $7T+5T=12T$
 - 得到 S_{12} 、 S_{13} 、 S_{14} 、 S_{15} 的时间延迟为 $9T+5T=14T$
 - 得到 C_{16} 的时间延迟为 $=5T+2(n-1)T=11T$;
 - 因为先行进位 P 和 G 并行生成，之后再需要 $2T$ 即可得到最高位进位位 C_4 ， C_8 ， C_{12} ， C_{16} ，参看四位先行进位电路需要 $2T$ （图3.6）
 - 采用串行进位，得到运算结果需要的时间延迟为 $(2n+4)T=(2 \times 16+4)T=36T$

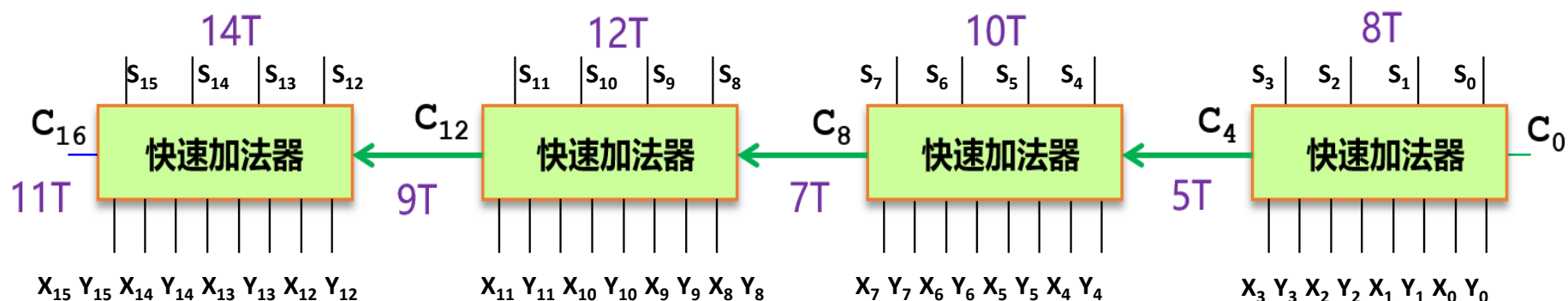


图3.7 16位加法器

- **16位加法器（组内并行、组间并行，多重分组，教材图3.8，可以进一步减少时间延迟）**

- 第1个（右下角）四位快速加法器： $C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$
- 定义： $G^* = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0$ $P^* = P_3 P_2 P_1 P_0$
- 则有： $C_4 = G^* + P^* C_0$ 对应可级联先行进位电路： $C_1 = G_0 + P_0 C_0$
- 类似的，可以得到 C_8 （第2个四位快速加法器的进位，对应可级联先行进位电路的 C_2 ）、 C_{12} （第3个四位快速加法器的进位，对应可级联先行进位电路的 C_3 ）的计算公式： $C_2 = G_1 + P_1 C_1$ $C_3 = G_2 + P_2 C_2$
- 图3.8中的四位快速加法器见图3.6、可级联先行进位电路见图3.5
- 图3.8时间延迟分析：**5T时间延迟得到四位快速加法器的 P^* 、 G^*** ，再经过**2T时间延迟得到可级联先行进位电路的 C_1 、 C_2 、 C_3** （ $7T=5T+2T$ （1T与门+1T或门）），再经过**5T时间（图3.6 2T+3T的异或门）延迟得到运算结果 $S_{15}.....S_0$** ；因此总的的时间延迟为**12T=5T+2T+5T**；注：**此时图3.6中与门异或门电路的3T已经计算完毕，不占用时间**

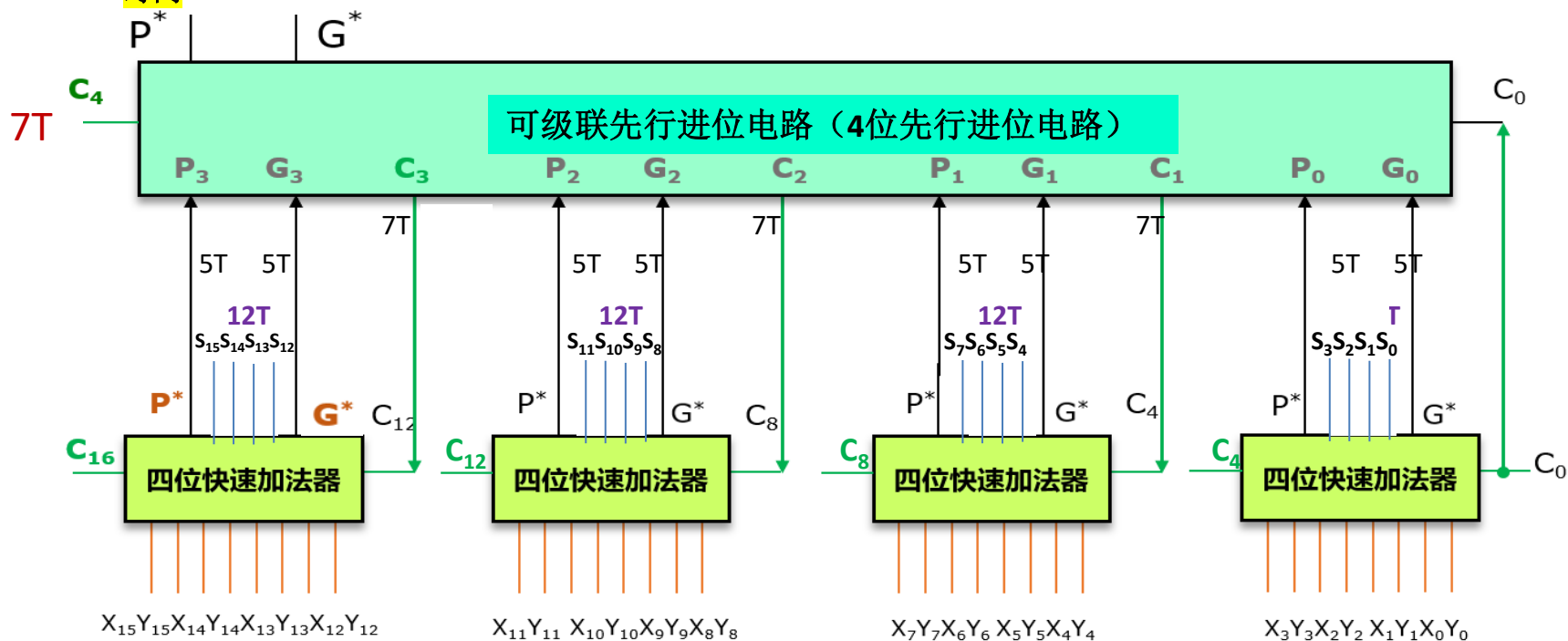


图3.8 16位加法器（组内并行、组间并行）

- **16位加法器时间延迟比较:**

- **16位加法器（串行，图3.3）:**

- 时间延迟

$$=(2n+4)T=(2*16+4)T=36T$$

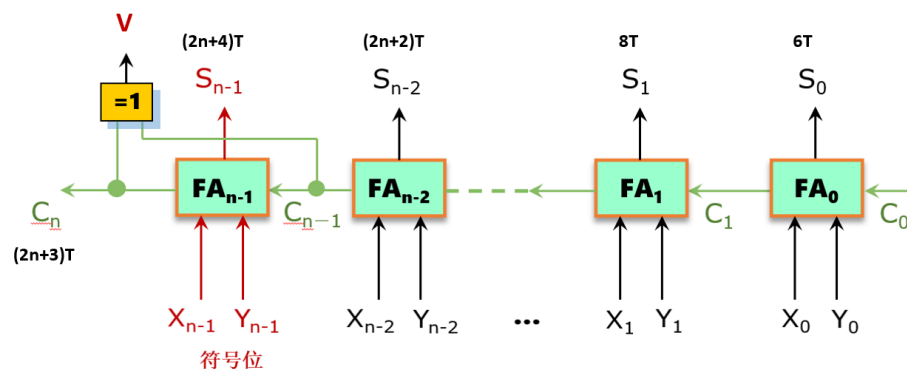


图3.3 多位串行加法器

- **16位加法器（组内并行、组间串行，图3.7）:**

- 时间延迟=14T

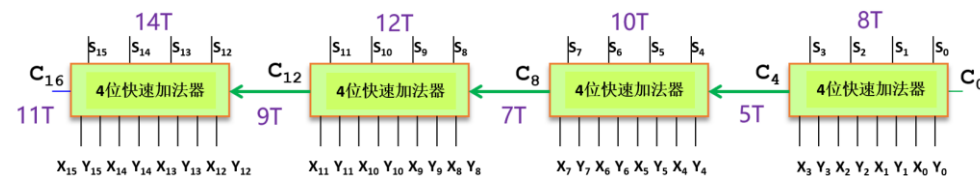


图3.7 16位加法器（组内并行、组间串行）

- **16位加法器（组内并行、组间并行，图3.8）:**

- 时间延迟=12T

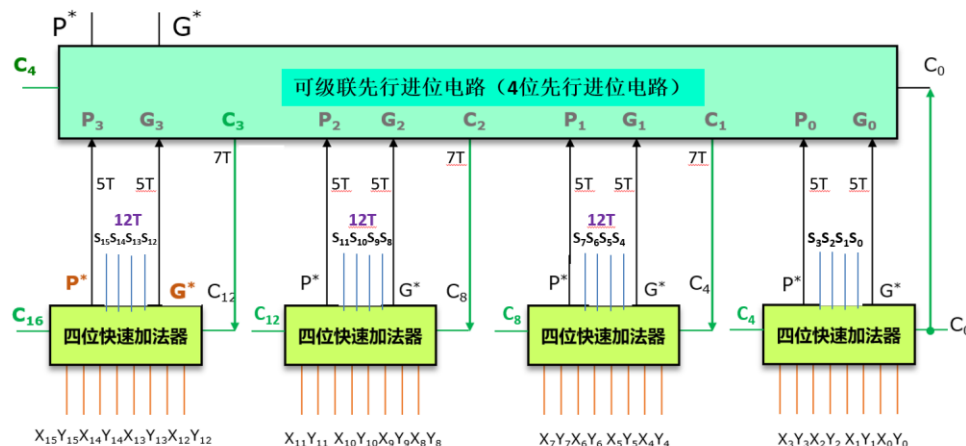


图3.8 16位加法器（组内并行、组间并行）

先行进位加法器

- 如何实现**32**位并行加法器？
 - 思路：类似于图3.7的方式，采用**2**个**16**位并行加法器**串联**（组内并行、组间串行）；**16**位并行加法器采用图3.8的形式
- 如何实现**64**位并行加法器？
 - 思路：
 - 方法1：类似于图3.7的方式，采用**4**个**16**位并行加法器**串联**（组内并行、组间串行）；**16**位并行加法器采用图3.8的形式
 - 方法2：类似于图3.8的方式，采用**4**个**16**位并行加法器**并联**（组内并行、组间并行）；**16**位并行加法器采用图3.8的形式

3.3 定点乘法运算

• 3.2.1 原码一位乘法

– 1、原码一位乘法运算方法

- (1) 乘积符号的确定

- 被乘数: $[x]_{\text{原}} = x_0 \cdot x_1 x_2 \dots x_n$

- 乘数: $[y]_{\text{原}} = y_0 \cdot y_1 y_2 \dots y_n$

- 乘积: $P = P_0 \cdot P_1 P_2 \dots P_{2n}$

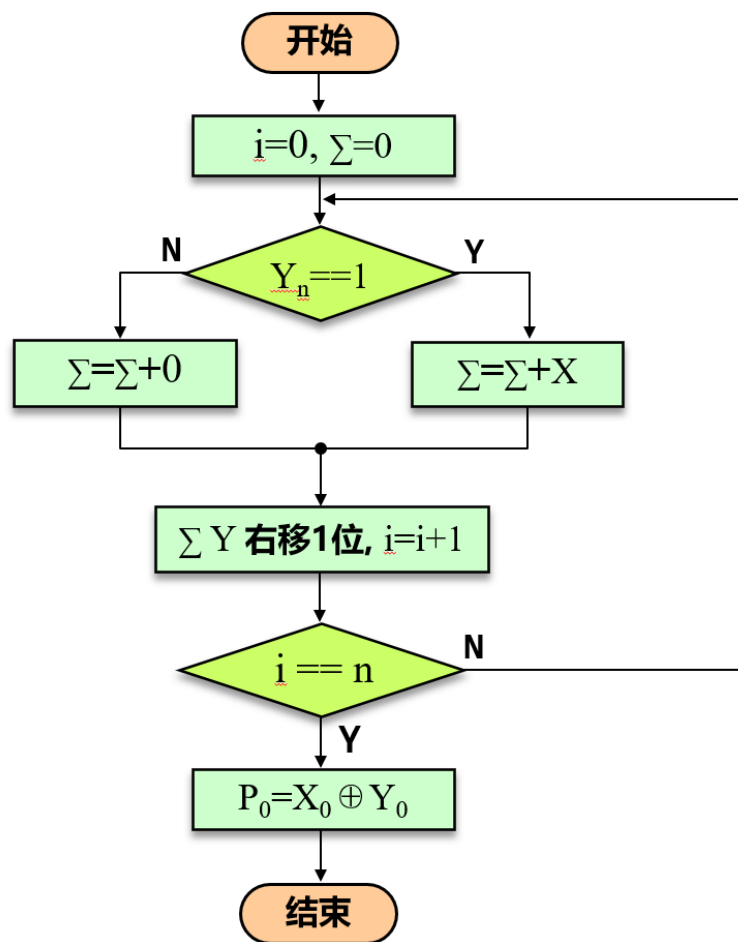
- 乘积的符号: $P_0 = x_0 \oplus y_0$

- (2) 乘积的数值

- $|P| = |x| \cdot |y|$

3.2.1 原码一位乘法

- 原码一位乘法算法流程图（教材图3.9）



$x=0.1101$, $y=0.1011$, 手工计算 $x \cdot y$

$$\begin{array}{r} 0.1101 \\ \times 0.1011 \\ \hline 1101 \\ 1101 \\ 0000 \\ + 1101 \\ \hline 0.10001111 \end{array}$$

图3.9 原码一位乘法算法流程图

3.2.1 原码一位乘法

• 例3.10: $x = 0.1101$, $y = -0.1011$, 用原码一位乘法求 $x \cdot y$

• 解:

- 乘积的符号 $P_0 = x_0 \oplus y_0 = 1$
- 乘积的绝对值 $= 0.1000\ 1111$ (具体见下面的计算过程)
- 乘积: $[x \cdot y]_{\text{原}} = 1.1000\ 1111$, $x \cdot y = -0.1000\ 1111$

	部分积	乘数 $ y $	说明
	00.0000	101 1	$P=0$
+	00.1101		$y_n=1, + x $
	00.1101	1011	
→	00.0110	110 1	逻辑右移一位
+	00.1101		$y_n=1, + x $
	01.0011	1101	
→	00.1001	111 0	逻辑右移一位
+	00.0000		$y_n=0, +0$
	00.1001	1110	
→	00.0100	111 1	逻辑右移一位
+	00.1101		$y_n=1, + x $
	01.0001	1111	
→	00.1000	1111	右移一位

乘数 $|y|$ 为 y (-0.1011) 的数值位 (**1011**), 部分积为**0** (双符号位小数)

根据 $|y|$ 的最后一位是**1**还是**0**, 执行 $+|x|$ 或 $+0$ 的操作

3.2.1 原码一位乘法

- 2、原码一位乘法的逻辑实现（教材图3.10）

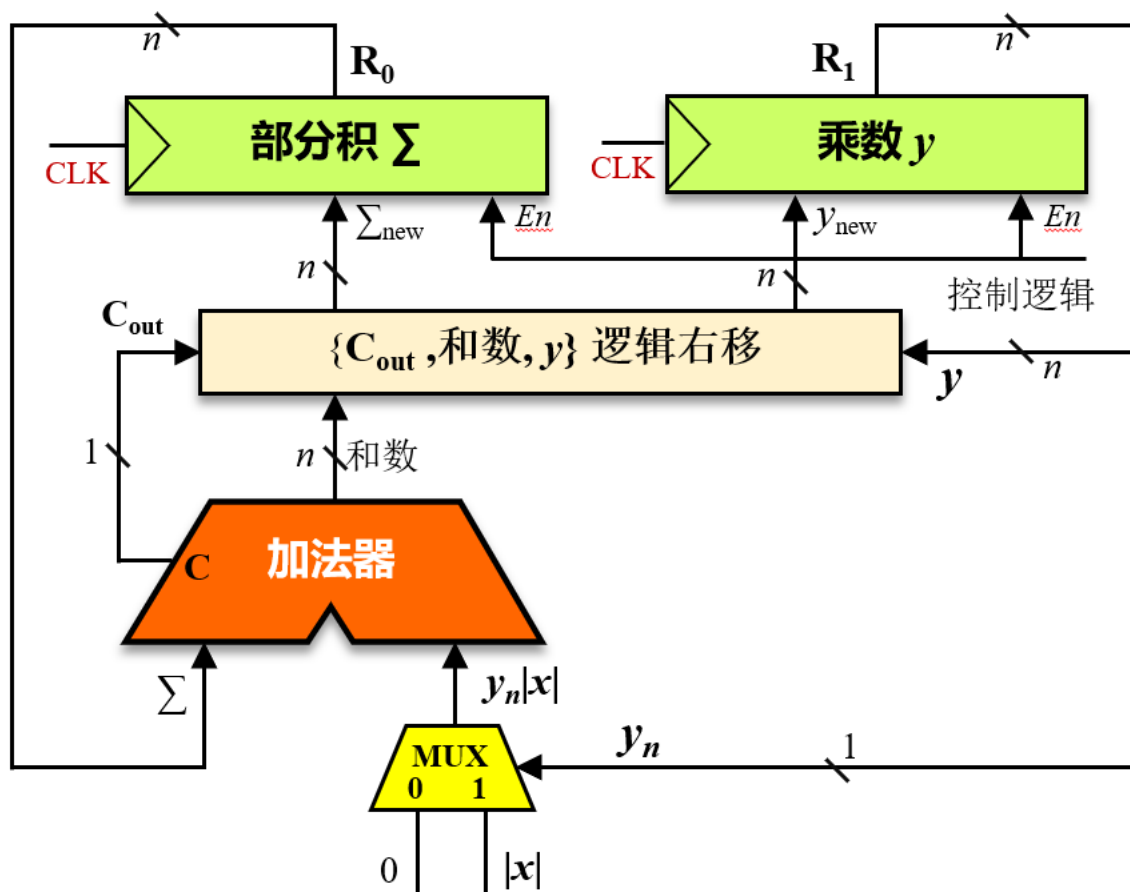
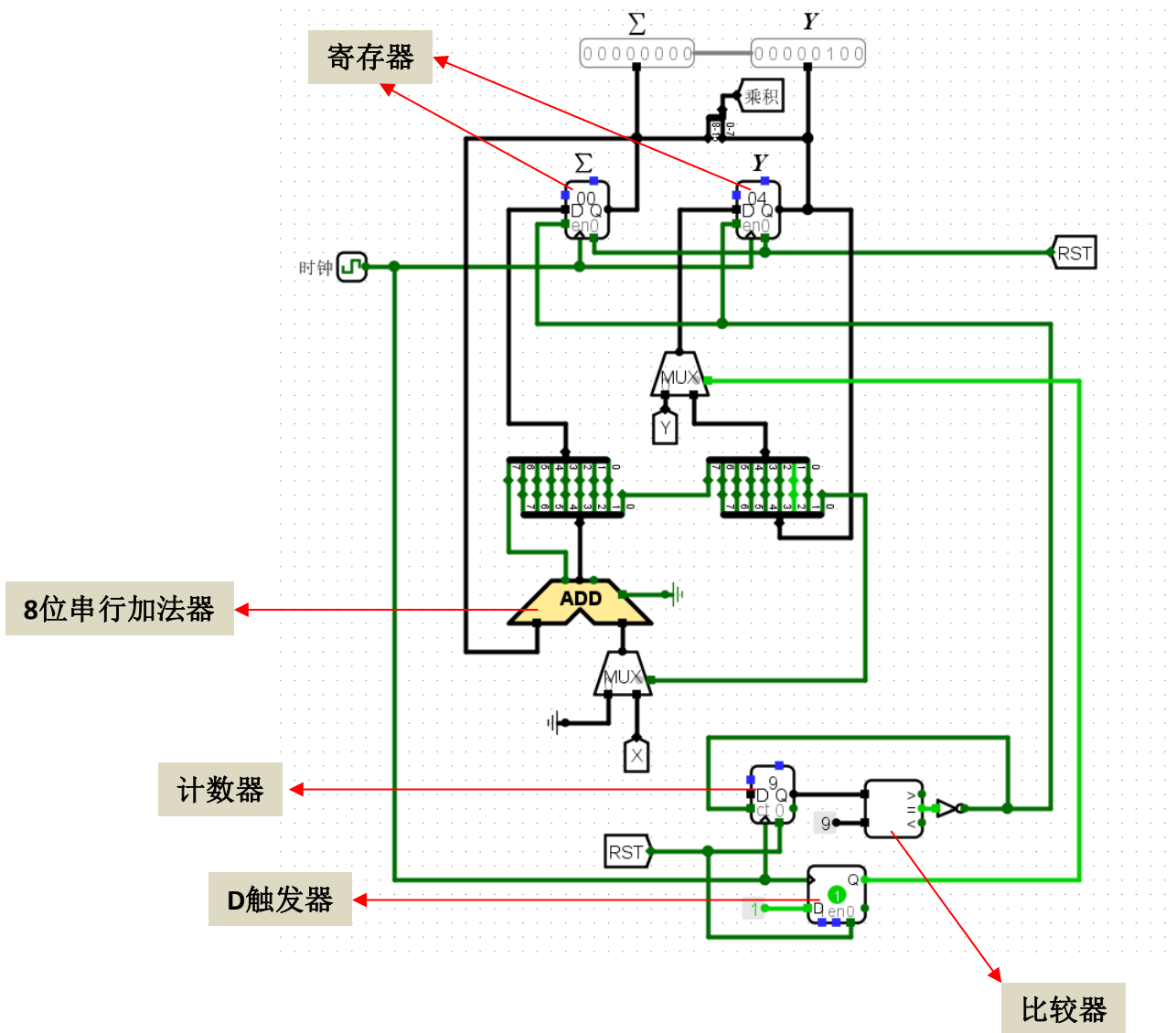


图3.10 原码一位乘法的逻辑实现

Logisim实现的8位无符号一位乘法器

设计文件“8位无符号一位乘法器.circ”（请从FTP下载）



输入：X、Y，8位无符号数

输出：乘积，16位无符号数

Σ：部分积，8位

RST：复位

利用该电路，可以实现8位
原码一位乘法器

输入：X、Y，8位有符号数
（原码）

输出：乘积，16位有符号数
（原码）

如何实现？请同学们思考（
第二次实验的内容）

3.2.1 原码一位乘法

- 原码二位乘法：每次完成二位的乘法，可以减少一倍的乘法运算时间（提高一倍的乘法运算速度）
- 原码二位乘法如何实现？请同学们查阅相关的资料

3.3.2 补码一位乘法

— 1、Booth算法

- (1) 被乘数 x 符号任意，乘数 y 符号为正数

$$- [x]_{\text{补}} \cdot [y]_{\text{补}} = [x \cdot y]_{\text{补}} = [x]_{\text{补}} \cdot y \quad \text{公式 (3-17)}$$

- (2) 被乘数 x 符号任意，乘数 y 符号为负数

$$- [x]_{\text{补}} \cdot [y]_{\text{补}} = [x \cdot y]_{\text{补}} = [x]_{\text{补}} \cdot 0.y_1y_2\cdots y_n - [x]_{\text{补}} \quad \text{公式 (3-20)}$$

- (3) 被乘数 x 符号任意，乘数 y 符号任意

$$- [x]_{\text{补}} \cdot [y]_{\text{补}} = [x \cdot y]_{\text{补}} = [x]_{\text{补}} \cdot 0.y_1y_2\cdots y_n - [x]_{\text{补}} \cdot y_0 \quad \text{公式 (3-21)}$$

3.3.2 补码一位乘法

- 2、补码一位乘法的算法（Booth算法，教材图3.11）

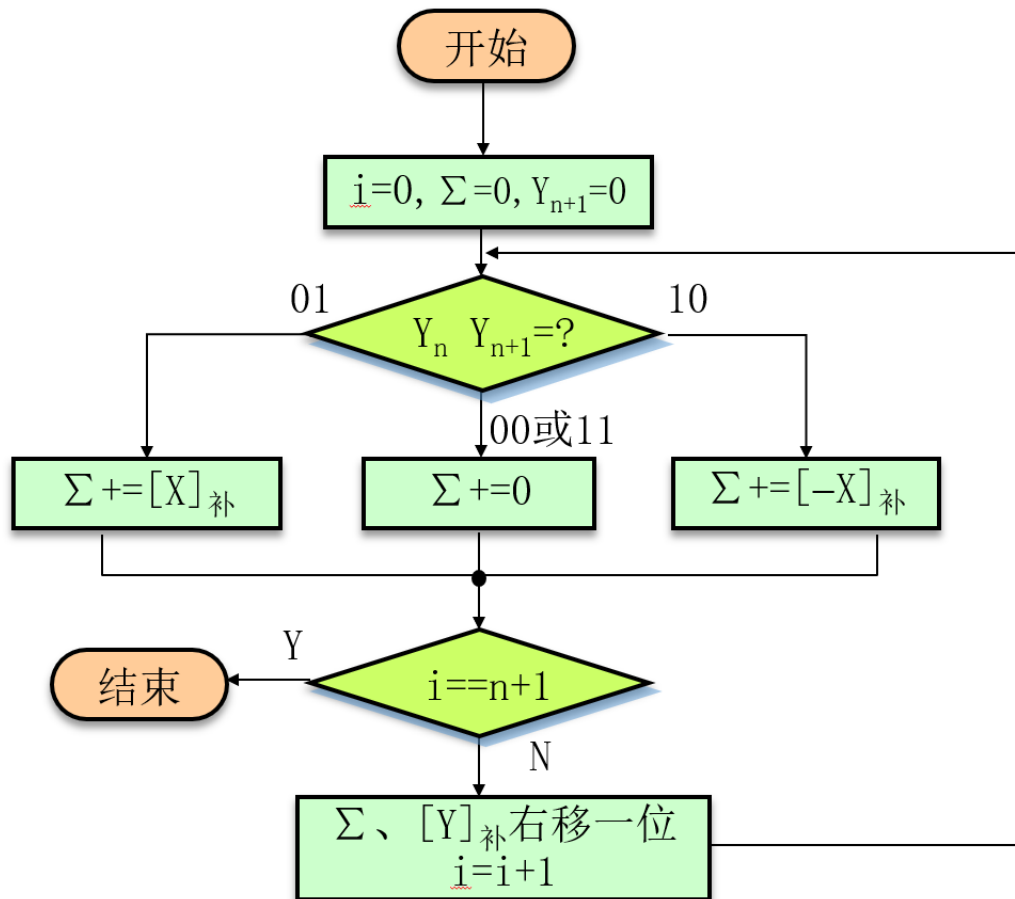


图3.11 补码一位乘法算法流程图

3.3.2 补码一位乘法

— 例3.11: $[x]_{\text{补}}=1.0111$,
 $[y]_{\text{补}}=1.0011$, 用补码一位乘法求 $[x \cdot y]_{\text{补}}$

— 解:

- $[-x]_{\text{补}} = 0.1000 + 0.0001 = 0.1001$ (取反加1)
- $[x \cdot y]_{\text{补}} = 0.0111\ 0101$ (具体见下面的计算过程)
- 验证: $x = -0.1001$, $y = -0.1101$, $x \cdot y = 0.0111\ 0101$

乘数 y 为补码值(1.0011), 去掉小数点(10011), 最后增加一位 $y_{n+1}=0$ (100110); 部分积为0(双符号位小数)

根据 y 的最后二位是01、10、00或11, 执行 $+[x]_{\text{补}}$ 、 $+[x]_{\text{补}}$ 、 $+0$ 的操作

	部分积	乘数 y	说明
+	00.0000 00.1001	1001 10	$P=0\ y_{n+1}=0$ $y_n y_{n+1}=10$, $+[x]_{\text{补}}$
→	00.1001 00.0100	100110	算术右移一位
+	00.0000	1100 11	$y_n y_{n+1}=11$, $+0$
→	00.0100 00.0010	110011	算术右移一位
+	11.0111	0110 01	$y_n y_{n+1}=01$, $+[x]_{\text{补}}$
→	11.1001 11.1100	011001	算术右移一位
+	00.0000	1011 00	$y_n y_{n+1}=00$, $+0$
→	11.1100 11.1110	101100	算术右移一位
+	00.1001	0101 10	$y_n y_{n+1}=10$, $+[x]_{\text{补}}$
	00.0111	0101	最后一步不移位

3.3.2 补码一位乘法

- 3、补码一位乘法的逻辑实现（Booth算法，教材图3.12）

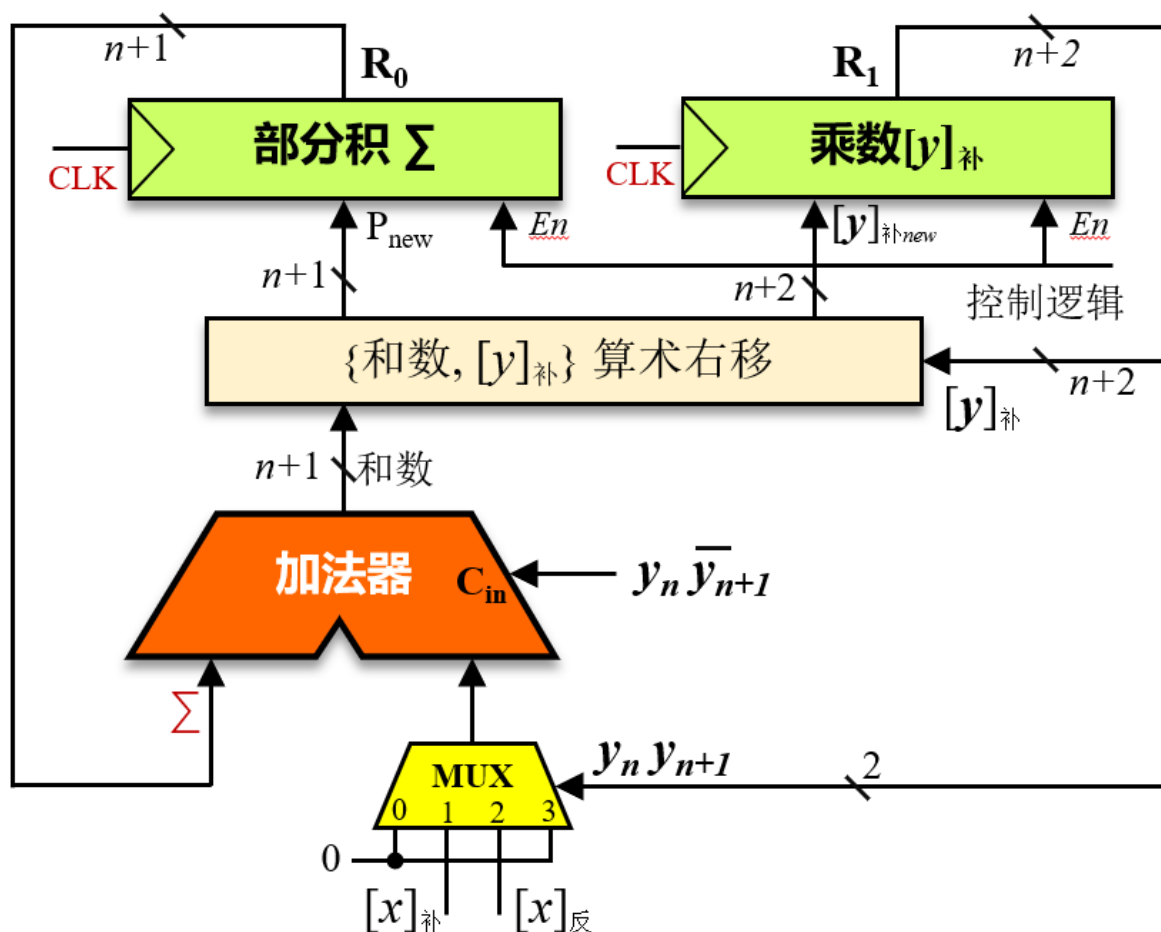
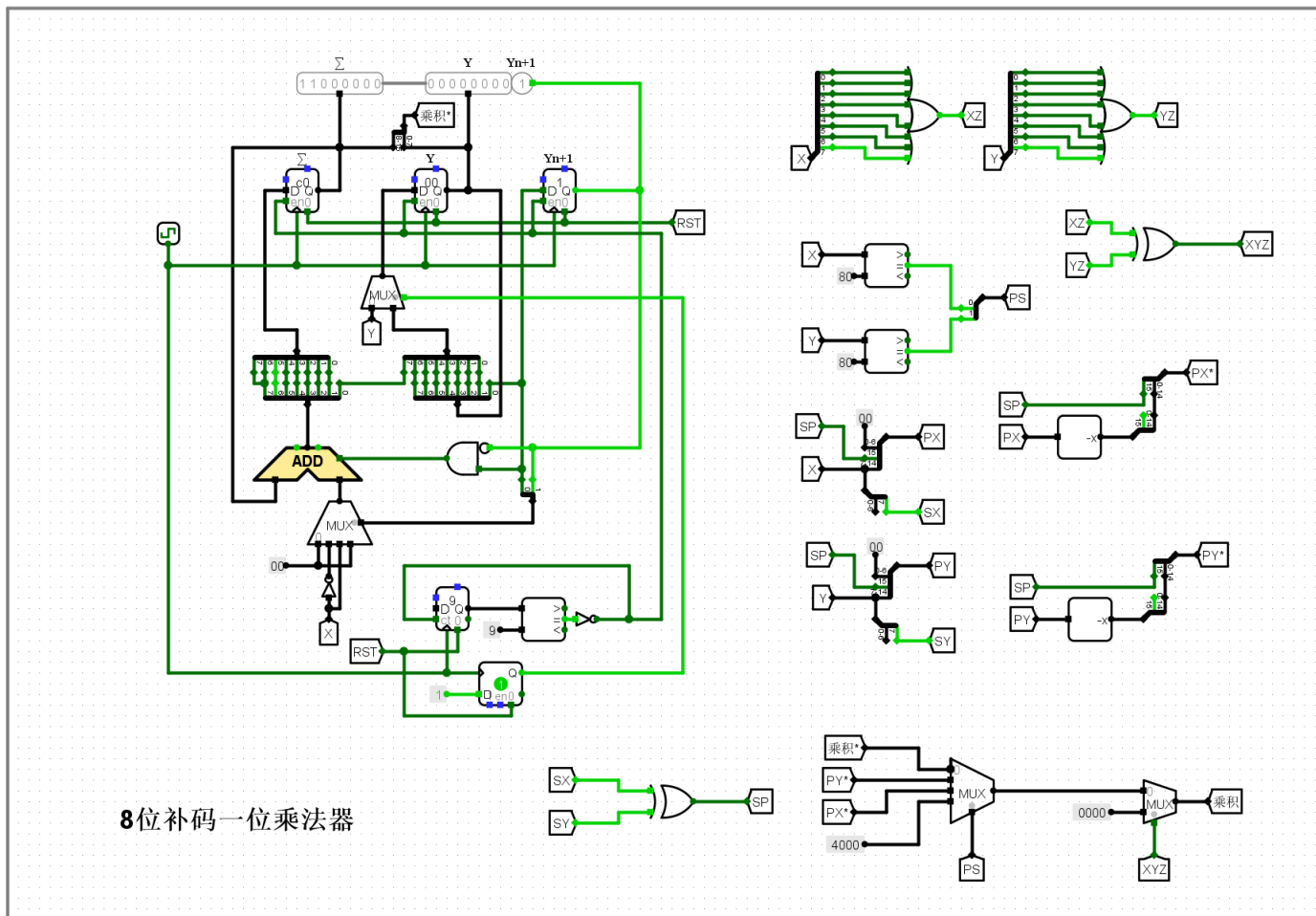
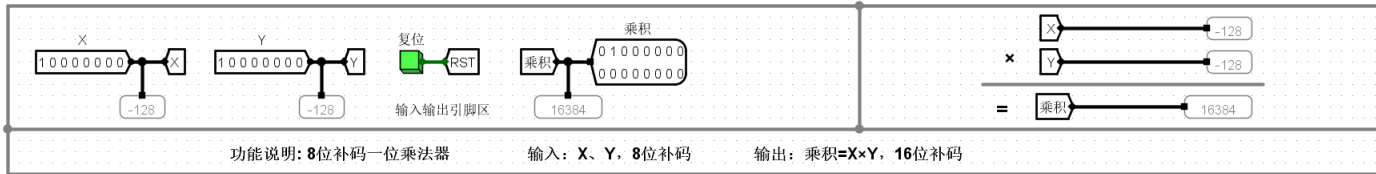


图3.12 补码一位乘法逻辑实现

Logisim实现的8位补码一位乘法器

设计文件“8位补码一位乘法器.circ”（请从FTP下载）

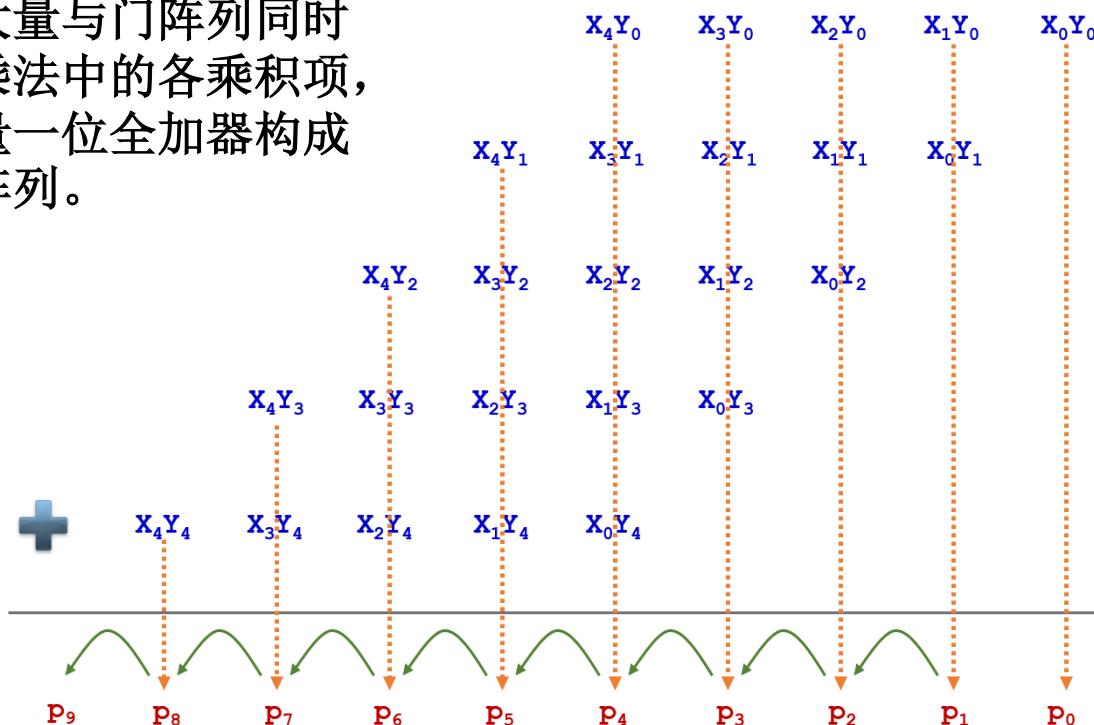


3.3.3 阵列乘法器

- 被乘数 $X = X_4X_3X_2X_1X_0$
- 乘数 $Y = Y_4Y_3Y_2Y_1Y_0$
- 乘积 $P = P_9P_8P_7P_6P_5P_4P_3P_2P_1P_0$
- 手工计算：

$$\begin{array}{r}
 X_4X_3X_2X_1X_0 \\
 \times \quad Y_4Y_3Y_2Y_1Y_0 \\
 \hline
 P_9P_8P_7P_6P_5P_4P_3P_2P_1P_0
 \end{array}$$

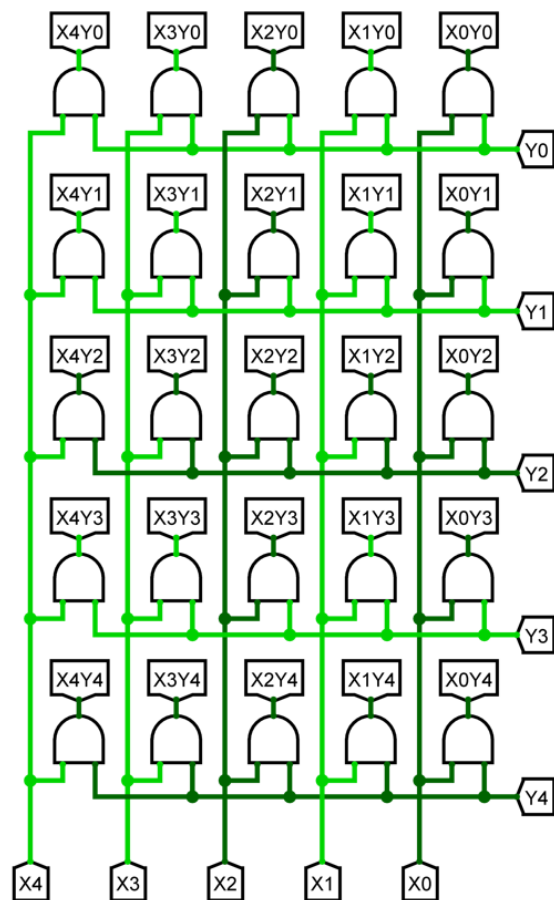
- 基本原理：采用手动乘法的方法，用大量与门阵列同时产生手动乘法中的各乘积项，同时将大量一位全加器构成等全加器阵列。



3.3.3 阵列乘法器

— （无符号）阵列乘法器的实现：

- **相加数产生电路**：1位乘法，得到25个乘积 $x_i \cdot y_j$ ，通过与门实现（25个与门）
- 行内并行，行间串行（进位链）

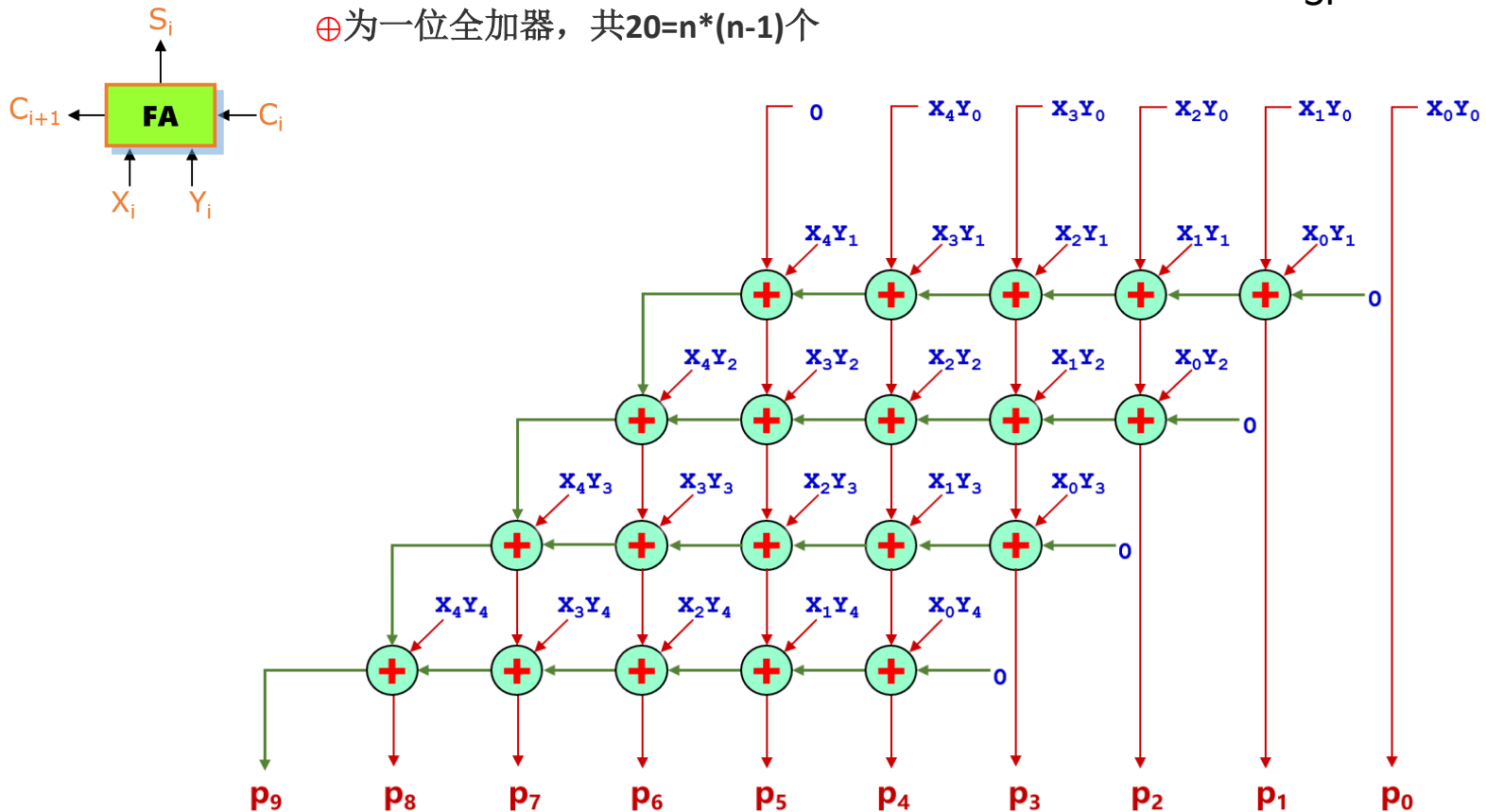


相加数产生电路

3.3.3 阵列乘法器

- 【HDL系列】乘法器(2)——阵列乘法器 - 知乎
- <https://zhuanlan.zhihu.com/p/119444283>
- 方法一：横向进位无符号阵列乘法器

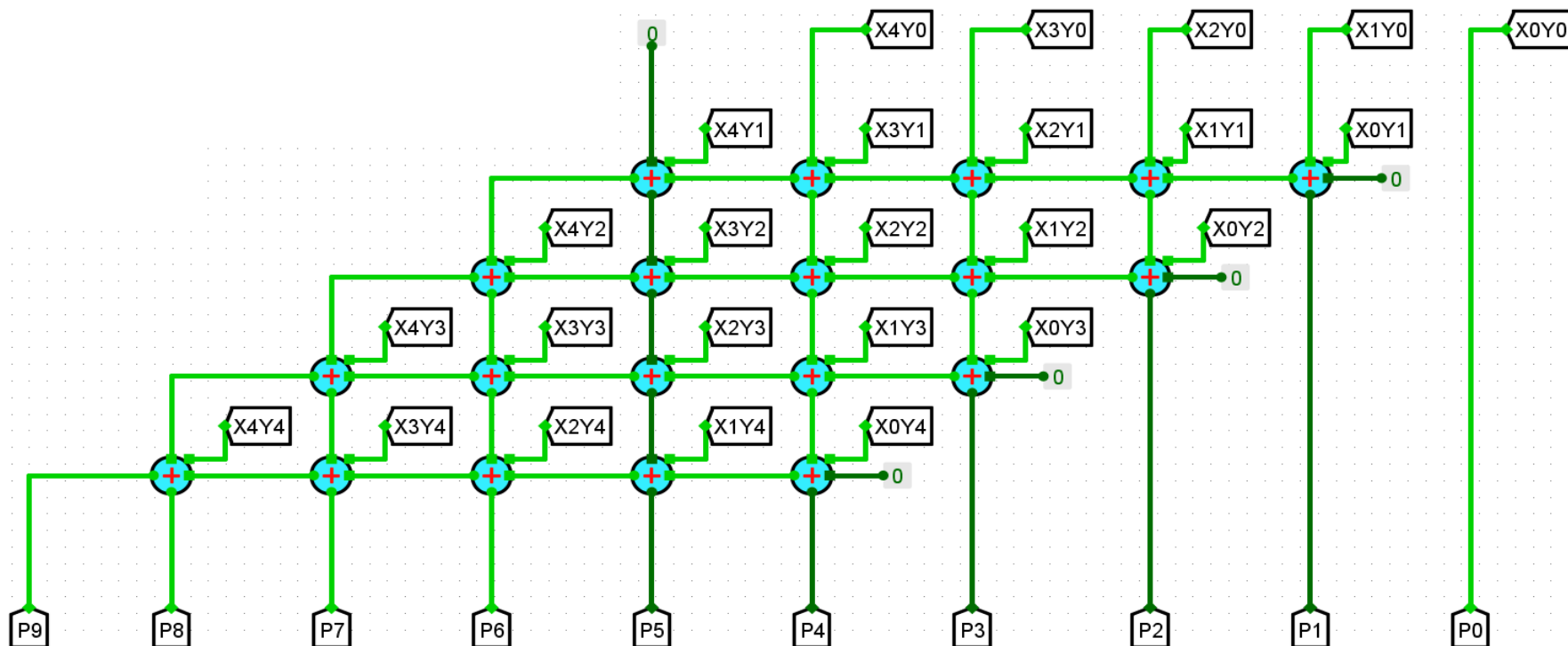
$\oplus$ 为一位全加器，共 $20 = n * (n - 1)$ 个



横向进位无符号阵列乘法器

Logisim实现的5位无符号阵列乘法器（横向进位）

设计文件“5位无符号阵列乘法器.circ”（请从FTP下载）



3.3.3 阵列乘法器

- 方法二：斜向进位无符号阵列乘法器（教材图3.13）
 - 图中 $\oplus$ 为一位全加器，共20个

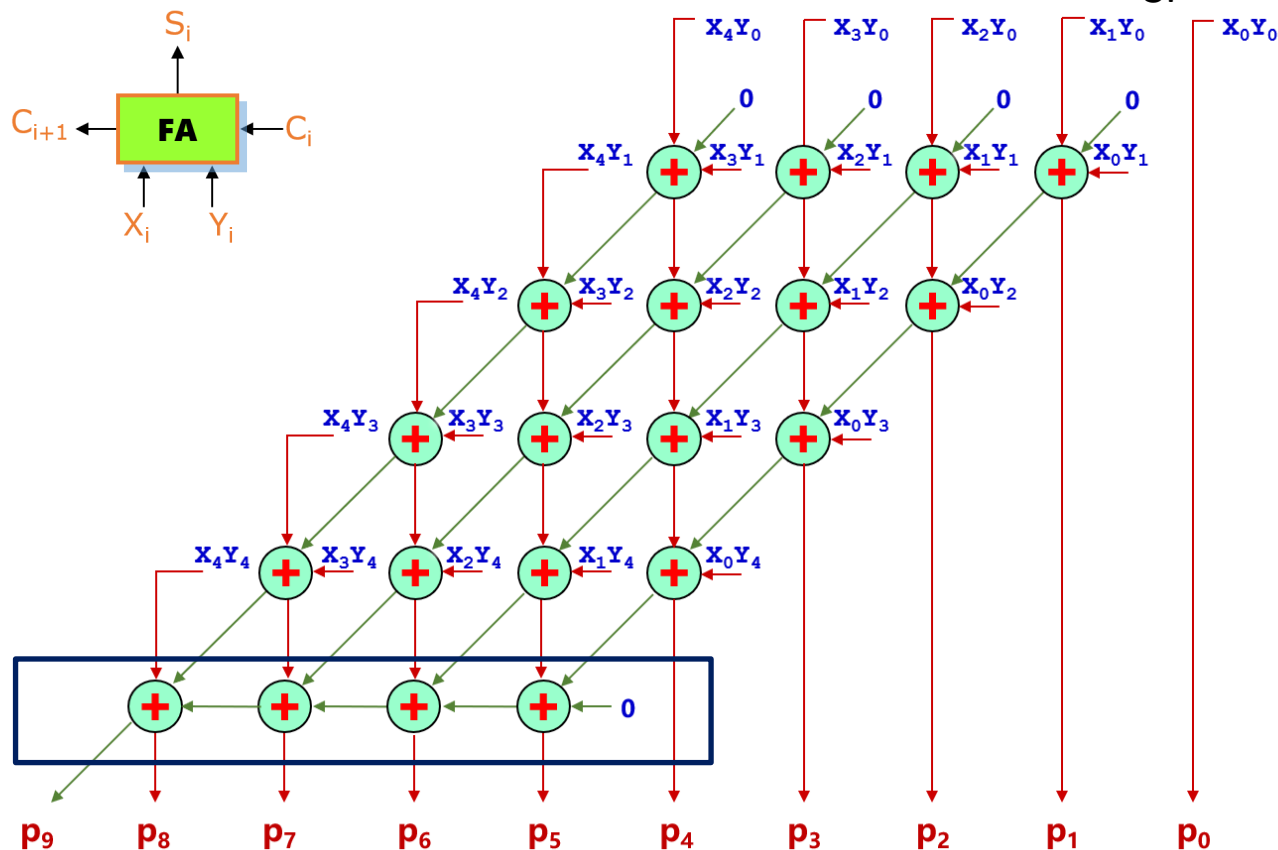
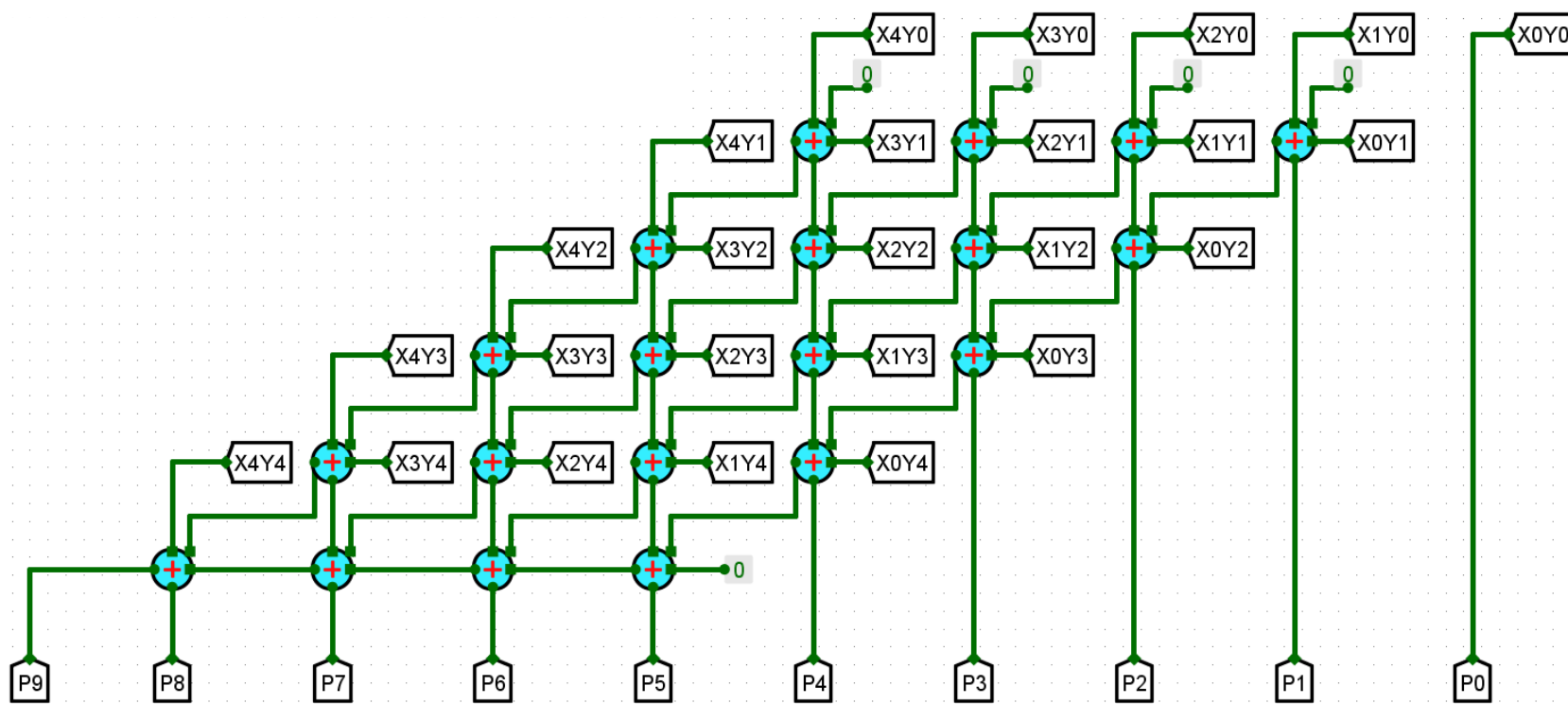


图3.13 斜向进位无符号阵列乘法器

Logisim实现的5位无符号阵列乘法器（斜向进位）

设计文件“5位无符号阵列乘法器.circ”（请从FTP下载）

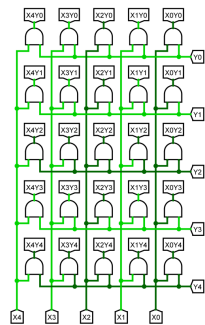
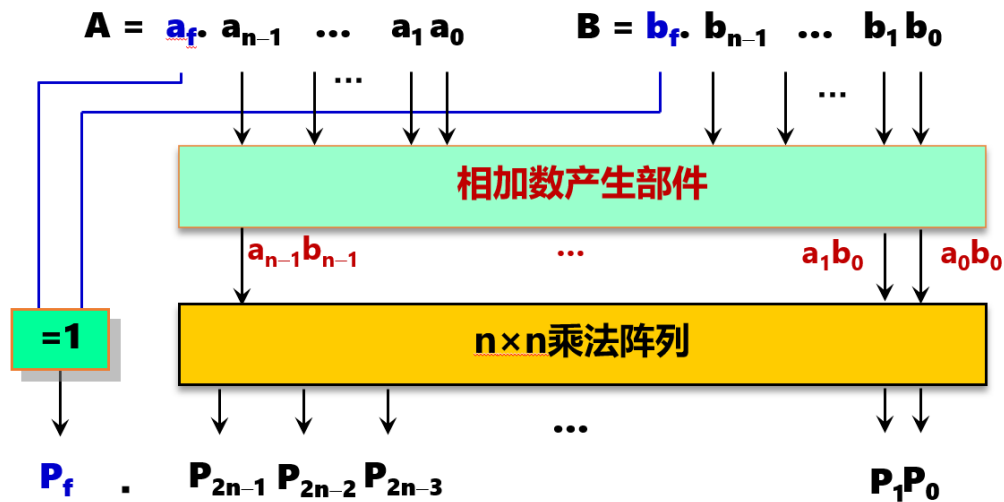


3.3.3 阵列乘法器

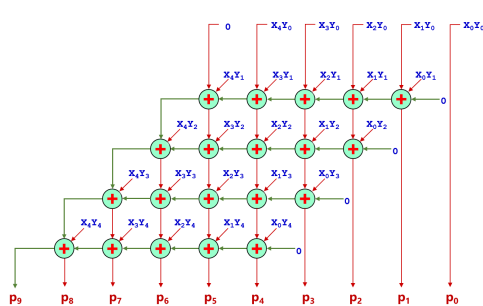
- 利用前面的5位无符号阵列乘法器，可以实现6位原码阵列乘法器
 - 乘数、被除数的数值位（5位）送5位无符号阵列乘法器，得到乘积的数值位
 - 乘数、被除数的符号位进行异或运算，得到乘积的符号位
 - 请同学们自行设计该电路（实验二的内容）

3.3.4 补码阵列乘法器

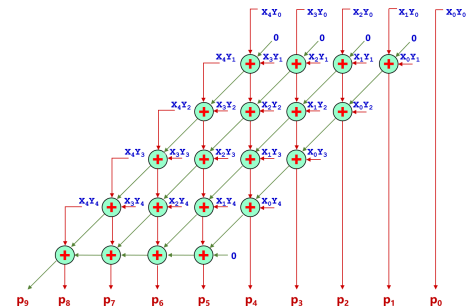
— 原码阵列乘法器:



相加数产生电路



$n \times n$ 乘法阵列 (横向进位)



$n \times n$ 乘法阵列 (斜向进位)

3.3.4 补码阵列乘法器

— 补码阵列乘法器：

- $a_f=1$ ，启动左边的 n 位求补器，将 a （负数）转换为绝对值
- $b_f=1$ ，启动右边的 n 位求补器，将 b （负数）转换为绝对值
- $a_f=0$ ， $b_f=0$ ，两个 n 位求补器不工作，直接将 a （正数）和 b （正数）送到相加数产生电路
- $a_f=1$ 、 $b_f=0$ ，或者 $a_f=0$ 、 $b_f=1$ ，结果为负数，启动 $2n$ 位求补器，得到乘积的补码
- $a_f=0$ 、 $b_f=0$ ，或者 $a_f=1$ 、 $b_f=1$ ，结果为正数， $2n$ 位求补器不工作，乘法阵列的输出就是乘积的补码

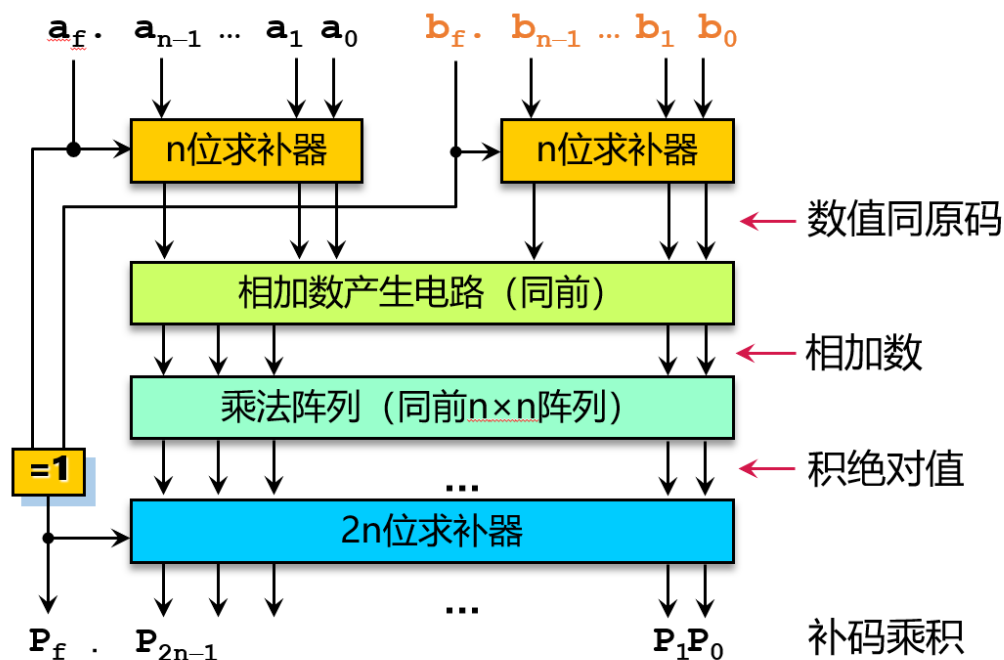
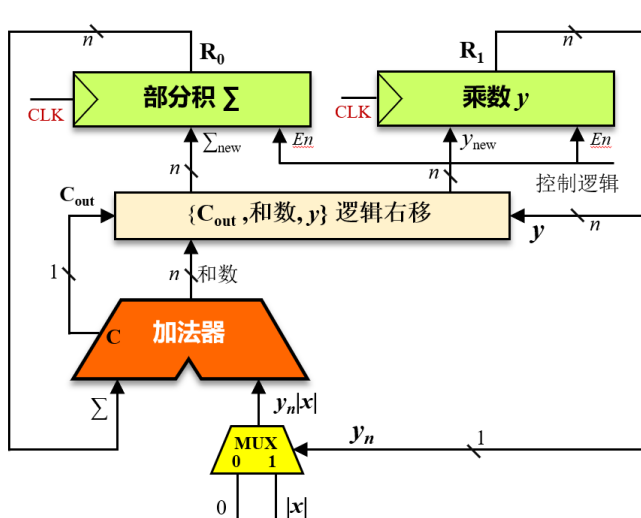


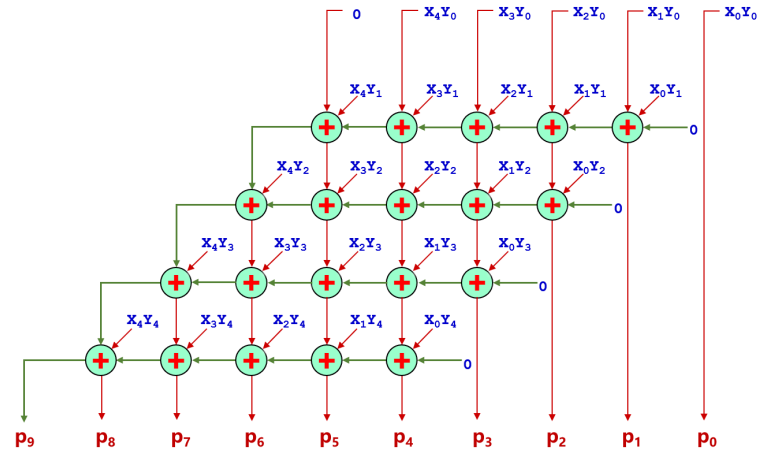
图3.14 补码阵列乘法器

3.3.5 乘法器性能优化

- 原码一位乘法：利用一个 n 位加法器进行 n 次累加运算实现 n 个位积的累加求和运算，时间复杂度为 $O(n^2)$
- 阵列乘法器：利用 $n(n-1)$ 个全加器实现快速的位积累加求和运算，时间复杂度为 $O(n)$
- Booth两位乘法（补码两位乘法）：时间复杂度为 $O(\log_2 n)$ ，具体算法请查阅相关文献



原码一位乘法

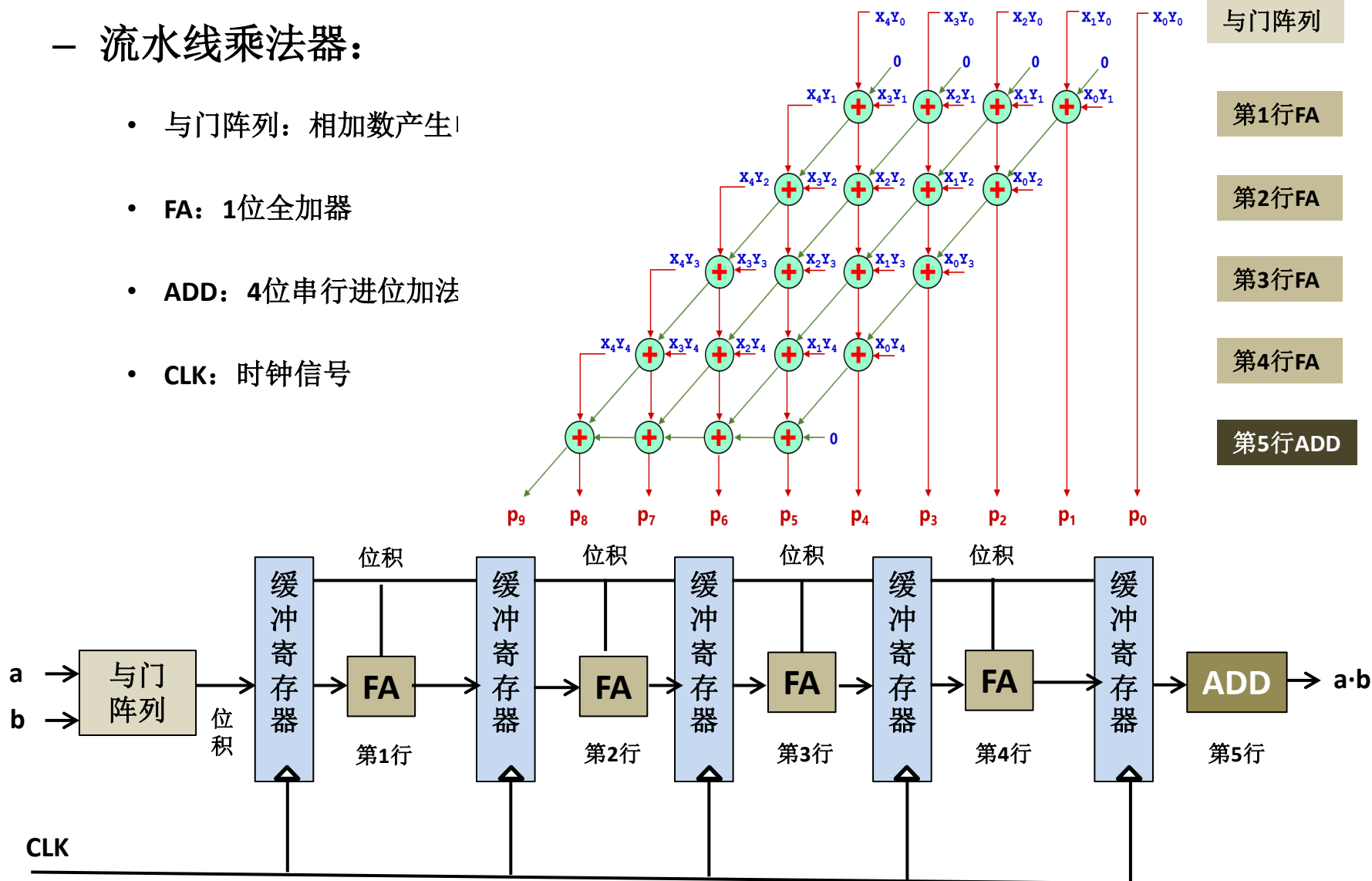


阵列乘法器

3.3.5 乘法器性能优化

— 流水线乘法器：

- 与门阵列：相加数产生
- FA：1位全加器
- ADD：4位串行进位加法
- CLK：时钟信号



3.4 定点除法运算

• 3.4.1 原码一位除法

- 被除数: $[x]_{\text{原}} = x_0.x_1x_2\dots x_n$
- 除数: $[y]_{\text{原}} = y_0.y_1y_2\dots y_n$
- 商: $[Q]_{\text{原}} = q_0.q_1q_2\dots q_n$
- 余数: $R = 0.r_1r_2\dots r_n$
- 商的符号: $q_0 = x_0 \oplus y_0$
- 商的数值: $|Q| = |x| \div |y|$
- 要求: $|x| < |y|$, 这样将不会发生溢出, 即商的绝对值 $|Q| < 1$

3.4.1 原码一位除法

— 手工除法运算：

- $x=0.1001001=73/128$, $y=0.1011=11/16$
- 商 $Q=0.1101=13/16$, 余数 $R=0.00000011=3/256$
- 验证: $x=Q \cdot y + R = (13/16) \cdot (11/16) + 3/256 = 146/256 = 73/128$

	0.1 1 0 1	商Q	操作说明
0.1 0 1 1	0.1 0 0 1 0 0 1 - 0.0 1 0 1 1 0.0 0 1 1 1 0 - 0.0 0 1 0 1 1 0.0 0 0 0 1 1 1 0.0 0 0 1 0 1 1 0.0 0 0 0 1 1 1 0 - 0.0 0 0 0 1 0 1 1 0.0 0 0 0 0 0 1 1	$q_0=0$ $q_1=1$ $q_2=1$ $q_3=0$ $q_4=1$	$x < y$ 够减 够减 不够减 够减
余数R	0.0 0 0 0 0 0 1 1		

3.4.1 原码一位除法

— 1、原码恢复余数法

- 例3.12: $[x]_{\text{原}}=1.1001$, $[y]_{\text{原}}=0.1011$, 求 $[x \div y]_{\text{补}}$

• 解:

- $|x|=0.1001$, $|y|=0.1011$, $[-|y|]_{\text{补}}=1.0101$
- 计算过程见下页
- 商的符号= x 的符号 $\oplus y$ 的符号= $1 \oplus 0=1$
- 余数的符号= x 的符号= 1
- 商 $|Q|=0.1101$, $[Q]_{\text{原}}=1.1101$
- 余数 $|R|=0.0000\ 0001$, $[R]_{\text{原}}=1.0000\ 0001$

验证:

$$x = -9/16, \quad y = 11/16$$

- 商 $Q = -0.1101 = -13/16$,
- 余数 $R = -0.0000\ 0001 = -1/256$
- $x = Q \cdot y + R =$
 $(-13/16) \cdot (11/16) + (-1/256) = -144/256 = -9/16$

3.4.1 原码一位除法

原码恢复余数法

	余数R	商Q	说明
	00.1001	0.0000	初始余数R= x , 商
Q=0 +[- y] _补	11.0101		减 y
← + y	11.1110 00.1011	0.0000	余数为负, 商0 加 y 恢复余数
← +[- y] _补	00.1001 01.0010 11.0101	0.0000	左移1位 减 y
← +[- y] _补	00.0111 00.1110 11.0101	0.0001 0.001	余数为正, 商1 左移1位 减 y
← +[- y] _补	00.0011 01.0110 11.0101	0.0011 0.011	余数为正, 商1 左移1位 减 y
+ y	11.1011 00.1011	0.0110	余数为负, 商0 加 y 恢复余数
+[- y] _补	00.0110 00.1100 11.0101	0.110	左移1位 减 y
	00.0001	0.1101	余数为正, 商1
余数 R =0.0000 0001 商 Q =0.1101			

余数R的初始值为
x的绝对值（双符
号位小数）

余数为负, 商0,
加|y|恢复余数,
左移1位, 减|y|

余数为正, 商1,
左移1位, 减|y|

3.4.1 原码一位除法

– 2、原码不恢复余数法

- 例3.13: $[x]_{\text{原}}=1.1001$, $[y]_{\text{原}}=0.1011$, 求 $[x \div y]_{\text{补}}$

- 计算过程见下页

- 运算结果中的C为进位标志, 并且商位与进位相同, $q=C$

- 运算结果同例3.12

- 不恢复余数法的规则:

- 余数为正, 商1, 余数左移1位, 减去除数
- 余数为负, 商0, 余数左移1位, 加上除数

- 一会儿做加法运算, 一会儿做减法运算, 也称为加減交替法

3.4.1 原码一位除法

	C	余数R	商Q	说明
		00.1001	0.0000	初始余数R= x , 商Q=0
$+[- y]_{\text{补}}$		11.0101		减 y
	0	11.1110	0.0000	余数为负, 商0
←		11.1100	0.000	左移1位
$+ y $		00.1011		加 y
	1	00.0111	0.0001	余数为正, 商1
←		00.1110	0.001	左移1位
$+[- y]_{\text{补}}$		11.0101		减 y
	1	00.0011	0.0011	余数为正, 商1
←		00.0110	0.011	左移1位
$+[- y]_{\text{补}}$		11.0101		减 y
	0	11.1011	0.0110	余数为负, 商0
←		11.0110	0.110	左移1位
$+ y $		00.1011		加 y
	1	00.0001	0.1101	余数为正, 商1

余数|R|=0.0000 0001 商|Q|=0.1101

余数R的初始值为
x的绝对值（双符
号位小数）

余数为负, 商0,
左移1位, 加|y|

余数为正, 商1,
左移1位, 减|y|

3.4.1 原码一位除法

— 3、原码不恢复余数法的逻辑实现（教材图3.16）

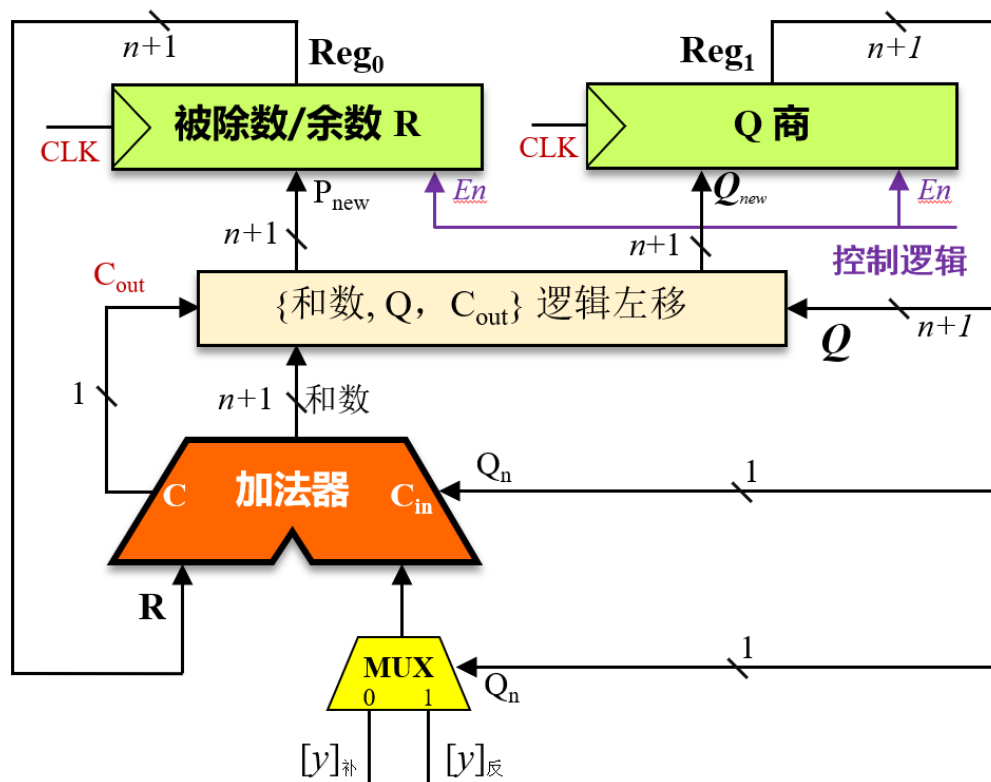


图3.16 原码不恢复余数法的逻辑实现

3.4.2 阵列除法器

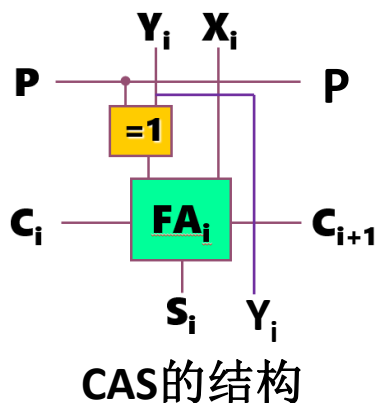
— 1、CAS的结构及其工作原理

- CAS: Controlled Adder Subtractor

- 输入: X_i 、 Y_i 、 C_i 、 P
- 输出: S_i 、 C_{i+1} 、 Y_i 、 P

- $P=0$, CAS做加法运算; $P=1$, CAS做减法运算

- $S_i = X_i \oplus (Y_i \oplus P) \oplus C_i$
- $C_{i+1} = (X_i + C_i)(Y_i \oplus P) + X_i C_i$



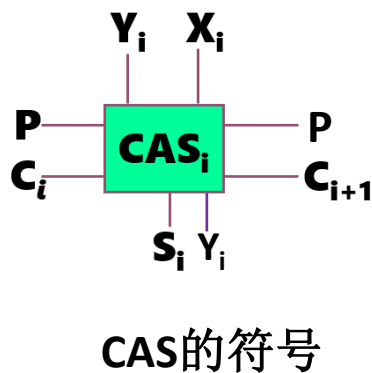
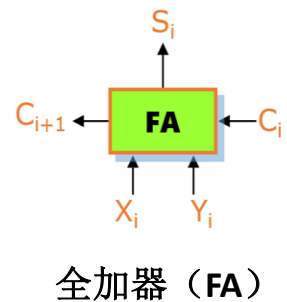
$$S_i = X_i \oplus Y_i \oplus C_i$$

$$C_{i+1} = X_i Y_i + (X_i \oplus Y_i) C_i$$

$$C_{i+1} = X_i Y_i + (X_i + Y_i) C_i$$

$$C_{i+1} = X_i Y_i + X_i C_i + Y_i C_i$$

$$= (X_i + C_i) Y_i + X_i C_i$$



– 2、阵列除法器的连接与运算

- 小数定点除的约束 $0 < |x| \leq |y|$
- 被除数 $X=0.X_1X_2X_3X_4X_5X_6X_7X_8$
- 除数 $Y=0.Y_1Y_2Y_3Y_4$
- 商 $Q=0.Q_1Q_2Q_3Q_4$
- 余数 $=0.000R_4R_5R_6R_7R_8$
- $P=1$ (CAS做减法运算, 并补码取反+1); 初始=1, 第一次进行比较, 判断溢出
- $C_{i+1}=Q_i$, 即进位位 C_{out} 为上商位, 作为下一行的 P , 控制每一行的加减运算
- Y_i 的斜向运算相当于手动的右移; 每一行的CAS构成串行加法/减法器

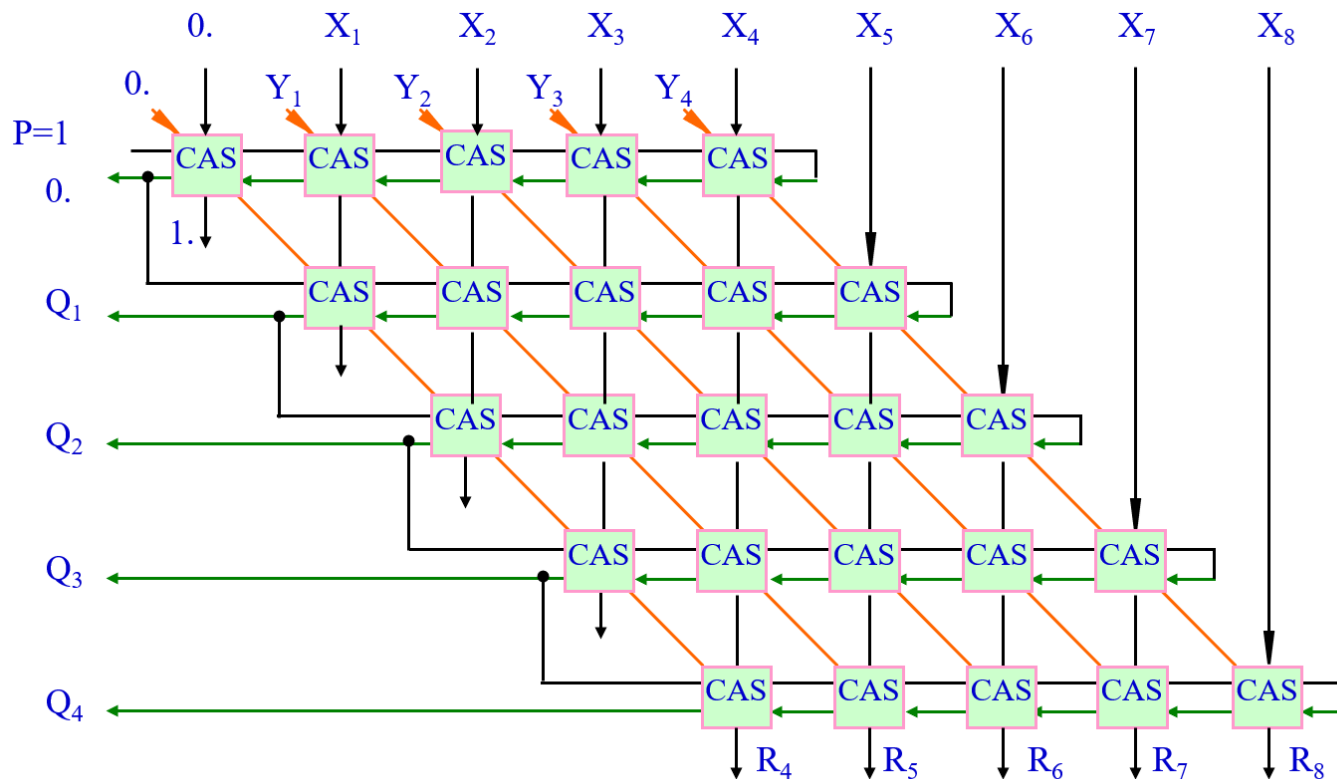
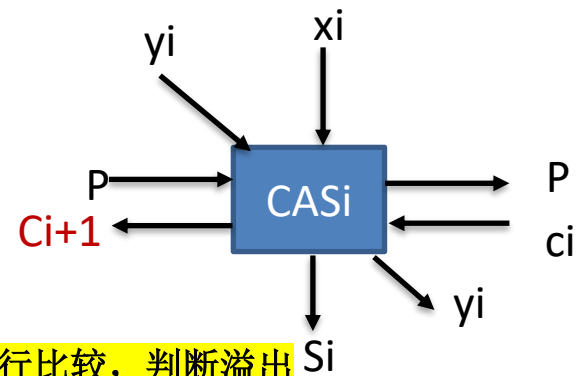


图3.18 阵列除法器

3.4.2 阵列除法器

– 3、阵列除法器时间延迟分析

- n 位串行可控加减法电路的时间延迟为（教材图3.4，见下图）： $(2n+7)T$
- n 位CAS阵列除法器需要 $(n+1)^2$ 个CAS，其时间延迟为： $[2(n+1)+7]T*(n+1)$
- 阵列除法器的时间复杂度为： $O(n^2)$
- 阵列乘法器的时间复杂度为： $O(n)$

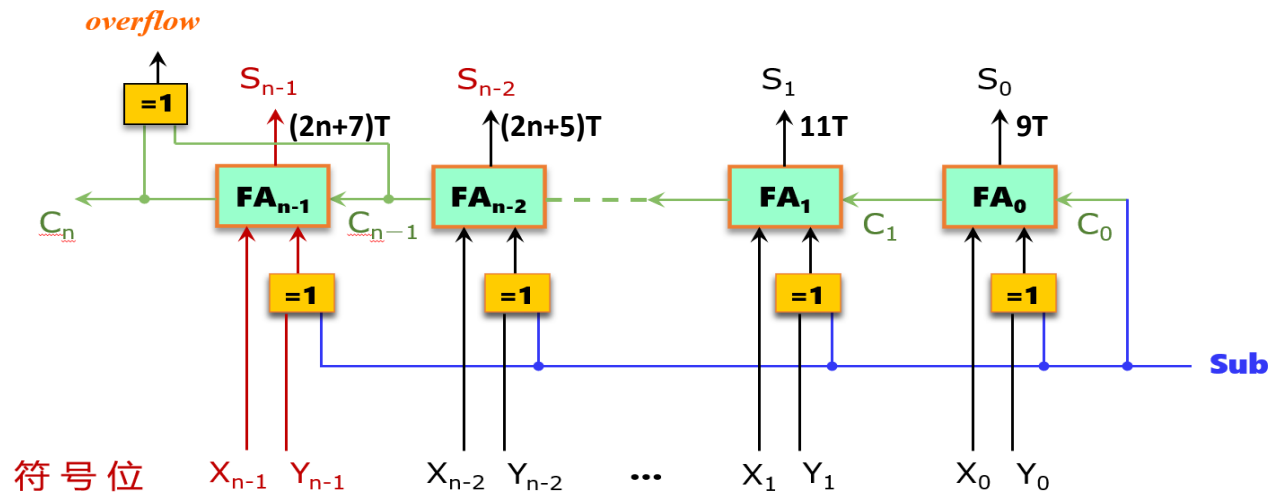


图3.4 n 位串行可控加减法电路

3.5 浮点运算

• 3.5.1 浮点加减法运算

– 1、阶码和尾数采用补码表示的浮点加减运算

- (1) 对阶：小阶向大阶对齐
- (2) 尾数运算：定点数的补码加减运算
- (3) 结果规格化：尾数的绝对值大于1，需要进行右规，尾数只需要右移1次，阶码加1；尾数的绝对值小于0.5，需要进行左规，尾数需要左移若干次，直到尾数的绝对值大于0.5（尾数每左移1次，阶码减1）
- (4) 舍入：尾数进行右规时（尾数右移），尾数的末尾会被丢掉，从而产生误差。舍入方法：末位恒置1法；0舍1入法
- (5) 溢出判断：阶码的符号位为01或10时，表示浮点数溢出了

3.5.1 浮点加减法运算

- 例3.14: 设 $x=2^{-101} \cdot (-0.101011)$, $y=2^{-010} \cdot (0.001110)$, 数的阶码为3位, 尾数为6位 (均不含符号位), 且都用补码表示, 按照补码浮点数运算步骤计算 $x+y$

• 解:

- 首先用补码形式表示浮点数 x 和 y (采用双符号位)

$$\bullet [x]_{\text{补}} = 11\ 011, 11.010101 \qquad [y]_{\text{补}} = 11\ 110, 00.001110$$

- (1) 对阶: 小阶向大阶对齐, x (阶码=-5) 向 y (阶码=-2) 对齐, x 的尾数右移3位

$$\bullet \text{对阶后的}[x]_{\text{补}} = 11\ 110, 11.\textcolor{red}{111}010(\textcolor{red}{101})$$

$$\begin{array}{r} [x]_{\text{补}} \quad 11\ 110, 11.111010\ (101) \\ + [y]_{\text{补}} \quad 11\ 110, 00.001110 \\ \hline \end{array}$$

- (2) 尾数运算 (求和):

$$[x+y]_{\text{补}} \quad 11\ 110, 00.001000\ (101)$$

- (3) 尾数规格化处理: 运算结果的尾数=00.001000(101), 为非规格化数 (尾数的绝对值太小), 需要左规, 左移2次, 变为规格化数00.100010(1), 阶码减2, 阶码变为11 100

- (4) 舍入处理: 舍入处理后的尾数为: 00.100011 (0舍1入法, 末位恒置1法)

- (5) 溢出判断: 阶码的双符号位相同 (11), 没有溢出

- 运算结果: $[x+y]_{\text{补}} = 11\ 100, 00.100011$, $x+y = 2^{-100} \cdot (0.100011)$

– 验证:

$$\begin{aligned} \bullet x &= (1/32) \cdot (-43/64), y = (1/4) \cdot (14/64), x+y = (-43/2048) + (112/2048) = 69/2048 \\ \bullet x+y &= 2^{-100} \cdot (0.100011) = (1/16) \cdot (35/64) = 70/2048, \text{相差 } 1/2048, \text{原因是舍入处理引起的误差} \end{aligned}$$

- 例3.15：设 $x=2^7 \cdot (25/32)$ ， $y=2^6 \cdot (-23/32)$ ，数的阶码为5位，尾数为7位（均含2位符号位），按照补码浮点数运算步骤计算 $x-y$ （采用0舍1入法）

• 解：

– 首先用补码形式表示浮点数 x 和 y （采用双符号位）

- $x=2^{111} \cdot (0.11001)$ $[x]_{\text{补}}=00\ 111, 00.11001$
- $y=2^{110} \cdot (-0.10111)$ $[-y]_{\text{补}}=00\ 110, 00.10111$

– (1) 对阶：小阶向大阶对齐， y （阶码=6）向 x （阶码=7）对齐， y 的尾数右移1位

- 对阶后的 $[-y]_{\text{补}}=00\ 111, 00.01011(1)$

– (2) 尾数运算（求和）：

$[x]_{\text{补}}$	00 111, 00.11001
$+ [-y]_{\text{补}}$	00 111, 00.01011(1)
$[x-y]_{\text{补}}$	00 111, 01.00100(1)

– (3) 尾数规格化处理：运算结果的尾数=01.00100(1)，为非规格化数（尾数的绝对值太大），需要右规，右移1次，变为规格化数00.10010(01)，阶码加1，阶码变为01 000

– (4) 舍入处理：舍入处理后的尾数为：00.10010（0舍1入法）

– (5) 溢出判断：阶码的双符号位为01，故发生溢出

– 运算结果： $[x-y]_{\text{补}}=01\ 000, 00.10010$

– 验证：

- $x=(128) \cdot (25/32)=100$ ， $y=(64) \cdot (-23/32)=-46$ ， $x-y=100-(-46)=146$ （超出-128 ~ +127范围）
- $x-y=2^8 \cdot (18/32)=144$ ，误差为2，是舍入处理导致的，误差为 $2^8 \cdot (0.0000001)=2$

3.5.1 浮点加减法运算

— 2、IEEE754浮点数的加减运算

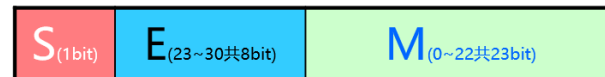
- IEEE754浮点数：阶码用移码表示；尾数用原码表示，且尾数的最高位隐藏（尾数=1.M）
- 结果规格化：
 - 若尾数=1.xx...x，则为规格化尾数
 - 若尾数=1x.xx...x，则表示尾数的绝对值大于2，需要进行1次右规，将尾数右移1次，阶码加1
 - 若尾数=0.xx...x，则表示尾数的绝对值小于1，需要进行左规，将尾数左移若干次，直到尾数=1.xx...x（即尾数的绝对值大于1），每左移1次，阶码减1
- 舍入处理：
 - 就近舍入
 - 朝 $+\infty$ 方向舍入
 - 朝 $-\infty$ 方向舍入
 - 朝0方向舍入
- 溢出判断：
 - 右规时，若阶码为全1（移码全1时为最大正数），则表示规格化上溢
 - 左规时，若阶码为全0（移码全0时为最小负数），则表示规格化下溢

3.5.1 浮点加减法运算

- 例3.16: 两个IEEE754单精度浮点数机器码X=00C0 0000H, Y=0080 0000H, 求X-Y

解:

32浮点数 float



- IEEE754单精度浮点数: 32位
- X=0000 0000 1100 0000 0000 0000 0000 0000=0 00000001
100000000000000000000000=1.1*2⁻¹²⁶
- Y=0000 0000 1000 0000 0000 0000 0000 0000=0 00000001
000000000000000000000000=1.0*2⁻¹²⁶
- E=阶码的真值+127, 阶码的真值=E-127=1-127=-126; 尾数=1.M
- (1) 对阶: X、Y的阶码相同, 不需要对阶
- (2) 尾数相减: 1.1-1.0=0.1
- (3) 结果规格化: 尾数相减后的0.1为非规格化数, 需要左移1次, 尾数变为1.0, 阶码减1, 阶码变为00000000 (全0)
- (4) 舍入处理: 没有舍入处理
- (5) 溢出判断: 阶码全0时为非规格化数据, 表示发生了规格化下溢
- IEEE754单精度非规格化浮点数: (-1)^S*0.M*2⁻¹²⁶, 这里: S=0, E=0, M=1
- X-Y=0.1*2⁻¹²⁶=2⁻¹²⁷=0 0000000 100000000000000000000000=0040 0000H

3.5.2 浮点乘法运算

- $X = 2^m \times M_x$
- $Y = 2^n \times M_y$
- $X \times Y = (2^m \times M_x) \times (2^n \times M_y) = 2^{m+n} \times (M_x \times M_y)$
- (1) 阶码相加: $m+n$
- (2) 尾数相乘: $M_x \times M_y$
- (3) 规格化舍入: 尾数相乘后的绝对值太小, 需要通过左规 (尾数左移), 将尾数变为规格化的数

3.5.3 浮点除法运算

- $X = 2^m \times M_x$
- $Y = 2^n \times M_y$
- $X \div Y = (2^m \times M_x) \div (2^n \times M_y) = 2^{m-n} \times (M_x \div M_y)$
- (1) 尾数调整：检查被除数（X）的尾数是否小于除数（Y）的尾数（绝对值大小比较）；如果大于，则将被除数的尾数右移1次，并将被除数的阶码加1（由于X、Y都是规格化的浮点数，因此只需要右移1次，就能实现被除数的尾数小于除数的尾数）
 - 例如：X=1.5×2^{xx...x}，Y=1.3×2^{xx...x}；调整后，X=0.75×2^{xx...x+1}，Y=1.3×2^{xx...x}
 - 例如：X=1.2×2^{xx...x}，Y=1.3×2^{xx...x}，不需要调整
- (2) 阶码求差：m-n
- (3) 尾数相除：M_x ÷ M_y
- (4) 规格化舍入：如果尾数相除后的商的绝对值太小，需要通过左规（尾数左移），将尾数变为规格化的数

3.6 运算器

• 3.6.1 定点运算器

– 1、运算器组成

• (1) 算术逻辑运算单元 (ALU)

– **ALU: Arithmetic and Logic Unit**, 算术逻辑单元; 算术逻辑运算单元是对定点数据进行加工处理的纯组合逻辑电路

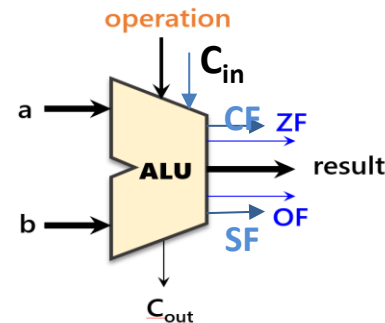
– **a、b**: 参与运算的两个操作数

– **C_{in}**: 进位输入

– **result**: 运算结果

– **C_{out}**: 进位输出

– **operation**: 决定做什么运算



– **CF: Carry Out Flag**, 加法进位输出, 或减法借位输出, 或左移操作的溢出位

– **ZF: Zero Flag**, 运算结果为0, 则ZF=1; 运算结果不为0, 则ZF=0

– **OF: Overflow Flag**, 有符号数运算溢出, 则OF=1; 不溢出, 则OF=0

– **SF: Sign Flag**, 运算结果为负数, 则SF=1; 为正数, 则SF=0

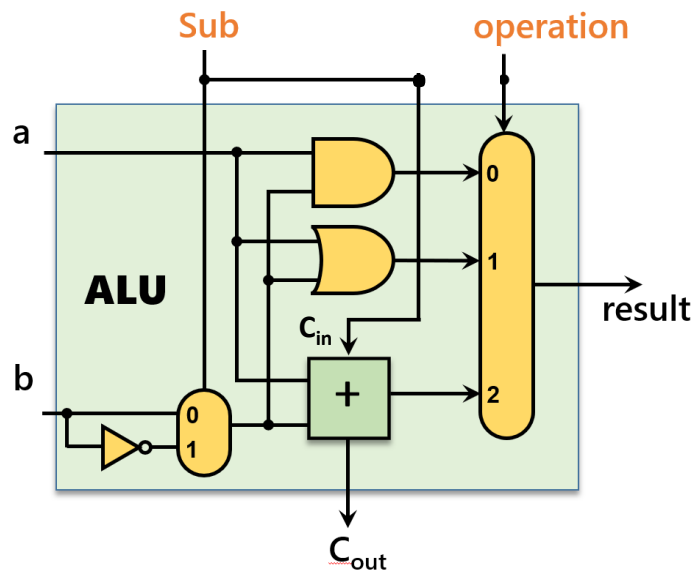
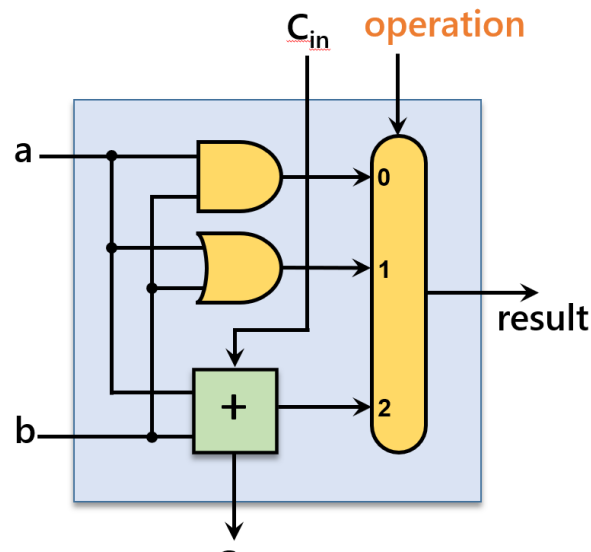
3.6.1 定点运算器

– 简单的1位ALU（右上角图）：

- operation=0, result=a & b
- operation=1, result=a | b
- operation=2, result=a + b + C_{in}

– 简单的1位ALU（右下角图）：

- Sub=0
 - operation=0, result=a & b
 - operation=1, result=a | b
 - operation=2, result=a + b
- Sub=1
 - operation=0, result=a & /b
 - operation=1, result=a | /b
 - operation=2, result=a + /b + 1 = a - b



3.6.1 定点运算器

- (2) 通用寄存器组
 - 也称为通用寄存器堆
- (3) 输入、输出数据选择控制
 - 多路选择器 (MUX)
- (4) 内部总线
 - IB: Internal Bus

3.6.1 定点运算器

– 2、运算器的基本结构

• (1) 单总线结构运算器

- LA、LB：缓冲寄存器（缓冲器）
- IB：内部总线

– 例如：ADD R_0, R_1 ; $(R_0) + (R_1) \rightarrow R_0$

– 第1个时钟周期： R_0 从通用寄存器经过内部总线IB送LA，再送A

 » $R_0 \rightarrow IB$, $IB \rightarrow LA$

– 第2个时钟周期： R_1 从通用寄存器经过内部总线IB送LB，再送B

 » $R_1 \rightarrow IB$, $IB \rightarrow LB$

– 第3个时钟周期：计算 $(R_0) + (R_1)$ ，并将运算结果经过内部总线IB送通用寄存器的 R_0

 » $AluOP=ADD$, $(R_0) + (R_1) \rightarrow IB$, $IB \rightarrow R_0$

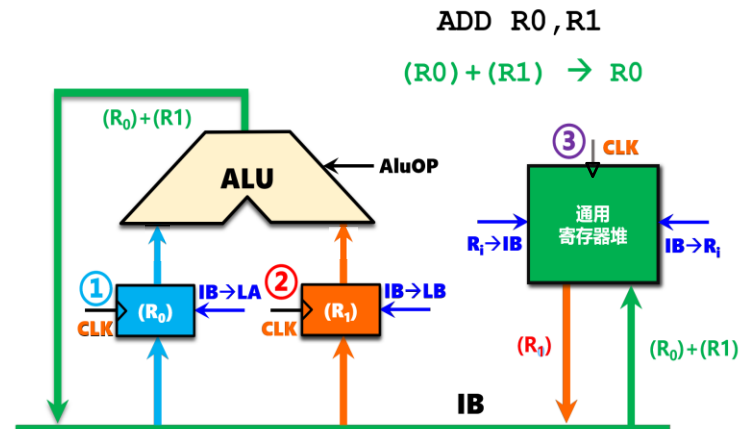
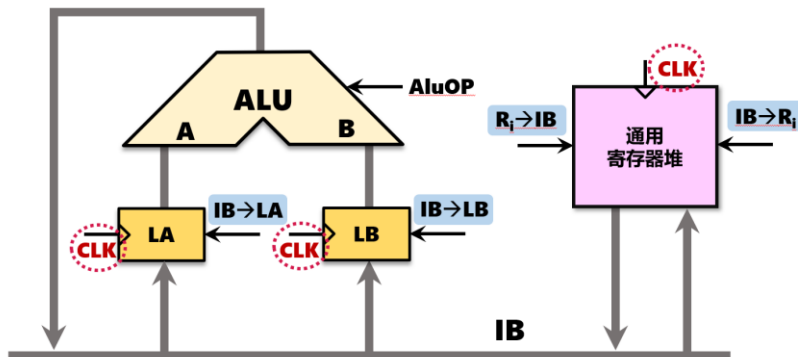


图3.20 单总线结构运算器

3.6.1 定点运算器

- (2) 双总线结构运算器
 - L: 缓冲寄存器 (缓冲器)
 - IB1: 内部总线1
 - IB2: 内部总线2
 - 通用寄存器堆有2个输出端、1个输入端
 - 例如: $ADD R_0, R_1 \quad ;(R_0) + (R_1) \rightarrow R_0$
 - 第1个时钟周期: R_0 从通用寄存器经过内部总线IB1送A, R_1 从通用寄存器结果内部总线IB2送B, 完成加法运算, 并将结果送L
 - $R_0 \rightarrow IB1, R_1 \rightarrow IB2, AluOP = ADD$
 - 第2个时钟周期: 运算结果从L经过内部总线IB1送通用寄存器的 R_0
 - $L \rightarrow IB1, IB1 \rightarrow R_0$
 - 只需要2个时钟周期, 而单总线结构的运算器需要3个时钟周期

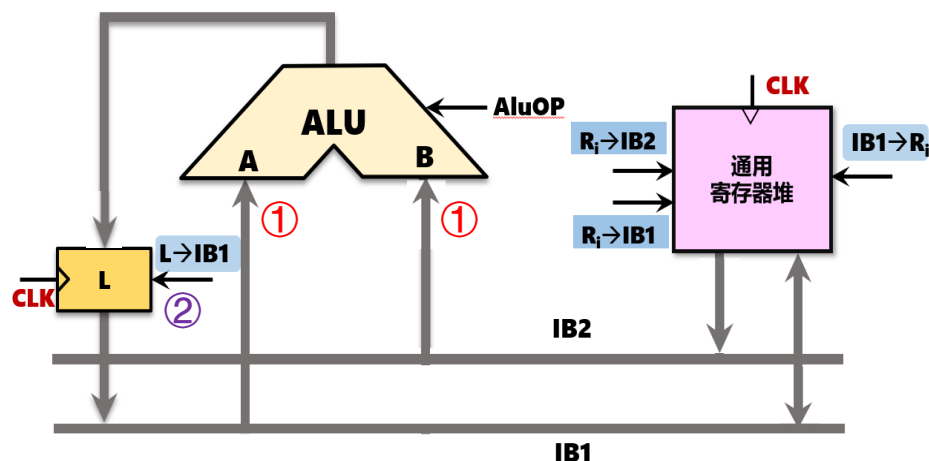


图3.21 双总线结构运算器

3.6.1 定点运算器

- (3) 三总线结构运算器
 - IB1: 内部总线1
 - IB2: 内部总线2
 - IB3: 内部总线3
 - 通用寄存器堆有2个输出端、1个输入端
 - 例如: **ADD R_0, R_1 ; $(R_0) + (R_1) \rightarrow R_0$**
 - 第1个时钟周期: R_0 从通用寄存器经过内部总线IB1送A, R_1 从通用寄存器经过内部总线IB2送B, 完成加法运算, 并将结果经过内部总线IB3送通用寄存器的 R_0
 - $R_0 \rightarrow IB1, R_1 \rightarrow IB2, ALU_{OP} = ADD, IB3 \rightarrow R_0$
 - 只需要1个时钟周期, 而单总线结构的运算器需要3个时钟周期、双总线结构的运算器需要2个时钟周期

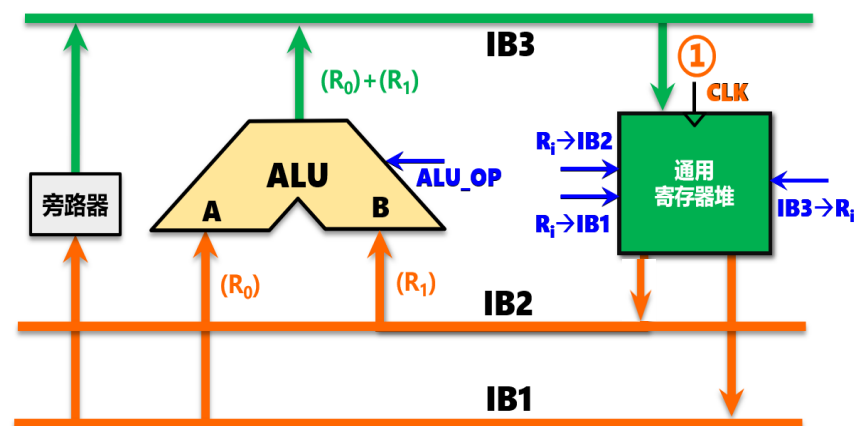
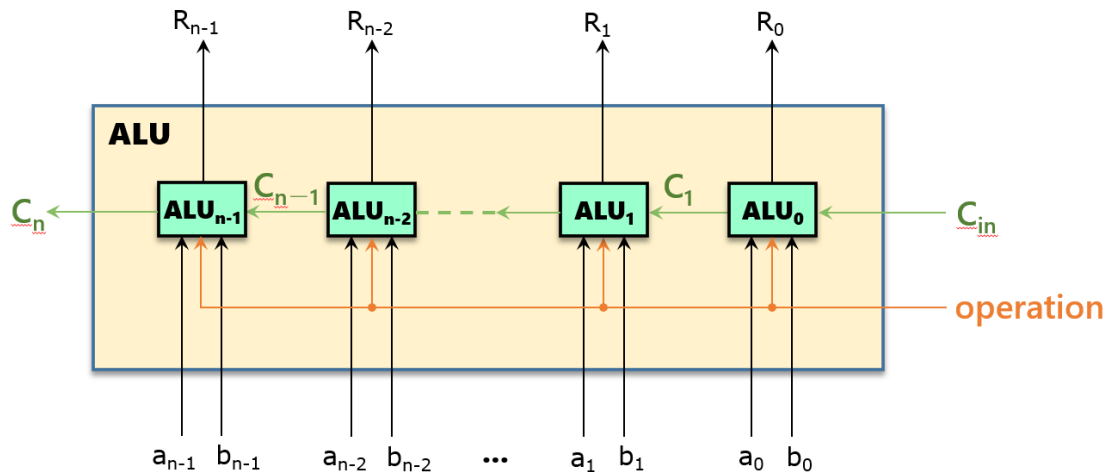
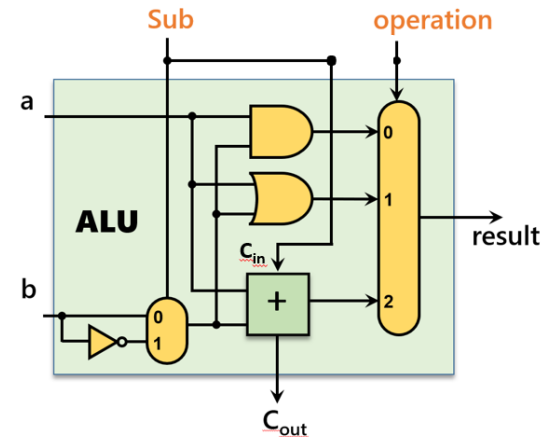


图3.22 三总线结构运算器

3.6.1 定点运算器

— 3、ALU设计

- 基本算术逻辑运算单元（1位ALU）
- 由n个1位ALU构成n位ALU



3.6.2 浮点运算

- 以浮点加减法为例，计算过程包括：对阶、尾数运算（尾数加减）、结果规格化、舍入处理、溢出判断
- 可以采用流水线的方式实现浮点加减法运算
- 浮点加减法流水线框图：

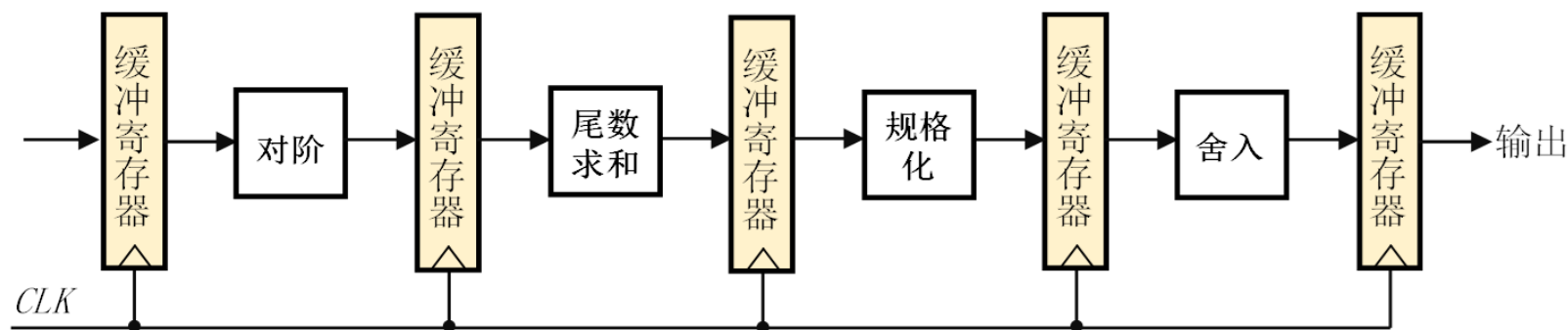


图3.26 浮点加减法流水线

本章小结

- C语言中的位运算、逻辑运算、移位运算、算术运算
- 补码加法、补码减法
- 溢出的概念：两个数相加（相减）的结果超出了补码的表示范围
- 溢出检测方法：
 - 方法一：根据操作数和运算结果的符号位是否一致
 - 方法二：根据最高数据位的进位和符号位的进位是否一致
 - 方法三：双符号位（变形补码）
- 全加器（FA）、半加器（HA）

- 串行加法器：也称行波进位加法器（**Ripple Carry Adder**）
 - 由n个FA构成n位串行加法器
- 可控加减法电路： $X - Y = X + /Y + 1$
- 先行进位加法器（**Carry Look-Ahead Adder**）：也称并行加法器、快速加法器
 - 进位生成函数G、进位传递函数P
 - 与门异或门电路：由 X_i 、 Y_i ，得到 G_i 和 P_i
 - 先行进位电路：由 G_i 和 P_i 以及 C_0 ，得到 C_n
 - 先行进位加法器（并行加法器、快速加法器）：与门异或门电路 + 先行进位电路
- 16位并行加法器：
 - 方法一：4个4位并行加法器串联而成（组内并行、组间串行）
 - 方法二：4个4位并行加法器并联而成（组内并行、组间并行）

- 原码一位乘法
 - 原码二位乘法：感兴趣的同学自己去查阅相关资料
- 补码一位乘法（**Booth**算法）
 - 补码二位乘法：感兴趣的同学自己去查阅相关资料
- （无符号）阵列乘法器
- 补码阵列乘法器
- 流水线乘法器
- 原码一位除法
 - 恢复余数法
 - 不恢复余数法（也称加减交替法）
- 阵列除法器
 - **CAS**：可控加减法单元

- 浮点加减法运算
 - ① 对阶
 - ② 尾数运算（尾数求和）
 - ③ 结果规格化
 - ④ 舍入（0舍1入法，末位恒置1法）
 - ⑤ 溢出判断
- IEEE754浮点数的加减运算
- 浮点乘法运算：阶码相加、尾数相乘、规格化舍入
- 浮点除法运算：阶码相减、尾数相除、规格化舍入
- 定点运算器
 - 状态标志：CF、ZF、SF、OF
 - 单总线结构运算器、双总线结构运算器、三总线结构运算器
- 浮点运算器
 - 浮点加减法流水线

实践训练（实验2）

- 在Logisim中构建8位可控加减法电路、4位先行进位电路、4位快速加法器、16位组内并行/组间并行加法器
- 在Logisim中设计5位无符号阵列乘法器，并利用该乘法器构造补码乘法器
- 在Logisim中设计8位原码一位乘法器和补码一位乘法器
- 在Logisim中设计能满足MIPS指令系统功能的32位多功能运算器